HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI THÀNH PHỐ HỒ CHÍ MINH
KHOA VIỄN THÔNG II

_____***_____

BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN

NĂM HỌC 2026-2027

35-SV-2026-VT2

THIẾT KẾ VÀ THI CÔNG BỘ ĐIỀU KHIỂN TRUNG TÂM TRONG HỆ THỐNG NHÀ THÔNG MINH

Giảng viên hướng dẫn: TS. Mai Hồng Anh
Sinh viên thực hiện: Trần Đức Thịnh – N23DCVT082

Thành phố Hồ Chí Minh, năm 2026

TÓM TẮT ĐỀ TÀI

Đề tài "Thiết kế và thi công bộ điều khiển trung tâm trong hệ thống nhà thông minh" tập trung nghiên cứu, xây dựng mô hình điều khiển các thiết bị điện trong nhà thông minh thông qua giọng nói và giao diện web. Mục tiêu của đề tài là kết hợp công nghệ Internet vạn vật (IoT), nhận dạng giọng nói và trí tuệ nhân tạo nhằm tạo ra một hệ thống có khả năng điều khiển thiết bị cục bộ, hỗ trợ hỏi đáp bằng giọng nói và giám sát trạng thái thiết bị từ xa.

Hệ thống được xây dựng trên nền tảng bốn vi điều khiển ESP32, đảm nhiệm các chức năng nhận dạng lệnh giọng nói, ghi âm và xử lý hỏi đáp, điều phối dữ liệu, cũng như điều khiển thiết bị. Công nghệ ESP-SR được sử dụng để nhận dạng từ khóa kích hoạt và các lệnh điều khiển ngoại tuyến. Đối với câu hỏi mở bằng tiếng Việt, hệ thống thực hiện ghi âm, gửi dữ liệu đến dịch vụ Gemini AI và phát câu trả lời thông qua công nghệ chuyển đổi văn bản thành giọng nói (Text-to-Speech – TTS). Các vi điều khiển trao đổi dữ liệu thông qua ESP-NOW, MQTT và UART tùy theo chức năng của từng khối.

Mô hình thực nghiệm tích hợp đèn LED RGB, quạt động cơ một chiều, cảm biến nhiệt độ và độ ẩm DHT11, cùng giao diện web phục vụ giám sát và điều khiển thiết bị. Ngoài ra, hệ thống có chức năng cấu hình kết nối Wi-Fi và MQTT thông qua trang web cấu hình. Quá trình thực hiện bao gồm phân tích yêu cầu, lựa chọn linh kiện, thiết kế kiến trúc phần cứng và phần mềm, triển khai firmware, xây dựng giao diện và kiểm thử các chức năng.

Kết quả triển khai cho thấy mô hình đã thực hiện được một số chức năng điều khiển thiết bị, tương tác bằng giọng nói, hỏi đáp bằng AI và giám sát qua web. Tuy nhiên, một số chức năng vẫn cần hoàn thiện, đặc biệt là xử lý lệnh đổi màu đèn bằng giọng nói và đánh giá định lượng về độ trễ, tỷ lệ nhận dạng. Báo cáo trình bày kiến trúc hệ thống, quá trình triển khai, kết quả kiểm thử, những hạn chế còn tồn tại và hướng phát triển trong tương lai.

Link github đề tài: https://github.com/tranthinhcv09/NCKH_trungtamdieukhienthietbi

Từ khóa: Smart Home, ESP32, ESP-NOW, MQTT, Gemini AI, ESP-SR, Text-To-Speech.

Mục lục

TÓM TẮT ĐỀ TÀI i

DANH MỤC HÌNH VẼ v

DANH MỤC BẢNG vi

DANH MỤC TỪ VIẾT TẮT vii

CHƯƠNG 1: GIỚI THIỆU 1

1.1 Lý do chọn đề tài . . . 1
1.2 Sản phẩm tương tự thực tế . . . 1
1.3 Mục tiêu của đề tài . . . 1
1.4 Đối tượng, phạm vi và giới hạn . . . 2
1.5 Phương pháp thực hiện . . . 3
1.6 Sơ lược bố cục báo cáo . . . 3

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 4

2.1 Tổng quan về IoT và nhà thông minh . . . 4
2.1.1 Giới thiệu về Internet of Things (IoT) . . . 4
2.1.2 Mô hình nhà thông minh và phân tầng xử lý . . . 4
2.2 Vi điều khiển và linh kiện sử dụng . . . 5
2.2.1 ESP32-S3 (MCU alexa và MCU recording) [1] . . . 5
2.2.2 ESP32 DevKit V1 (MCU center) [1] . . . 5
2.2.3 ESP32 28 chân (MCU control) [1] . . . 6
2.2.4 Micro INMP441 . . . 6
2.2.5 Mạch khuếch đại MAX98357A và Loa [2] . . . 6
2.2.6 Module thẻ nhớ SD và thẻ SD 4GB . . . 6
2.2.7 Cảm biến nhiệt độ, độ ẩm DHT11 [3] . . . 6
2.2.8 Nút cảm ứng điện dung TTP223B [4] . . . 6
2.2.9 MOSFET AOD4184 và Động cơ DC [5] . . . 7
2.2.10 Đèn LED RGB (HW-179) . . . 7
2.3 Chuẩn giao tiếp và giao thức truyền thông . . . 7
2.4 Nhận dạng giọng nói và AI . . . 8

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG 9

3.1 Phân tích yêu cầu . . . 9
3.2 Kiến trúc tổng thể . . . 10
3.3 Thiết kế phần cứng . . . 11
3.3.1 Danh mục linh kiện . . . 11
3.3.2 Sơ đồ chân . . . 12
3.3.3 Lưu ý thiết kế . . . 12
3.4 Tổng quan phần mềm . . . 13

3.5 Firmware MCU alexa . . . 13
3.5.1 Luồng nhận dạng giọng nói . . . 13
3.5.2 Danh sách lệnh . . . 14
3.5.3 Mạng và gửi lệnh . . . 15
3.6 Firmware MCU recording . . . 15
3.6.1 Chuỗi hỏi–đáp . . . 15
3.6.2 Ghi âm . . . 16
3.6.3 Tải lên máy chủ AI . . . 16
3.6.4 Hàng đợi đọc câu trả lời [6] . . . 16
3.6.5 Lệnh UART nhận từ MCU alexa . . . 17
3.7 Firmware MCU center . . . 17
3.7.1 Hoạt động chính . . . 17
3.7.2 Đọc DHT11 . . . 18
3.7.3 Trang web cấu hình . . . 18
3.7.4 Nút bấm GPIO15 . . . 19
3.8 Firmware MCU control . . . 19
3.8.1 Điều khiển bằng cảm ứng . . . 19
3.8.2 Bộ phân tích lệnh . . . 19
3.8.3 Điều khiển quạt bằng PWM . . . 19
3.8.4 Tự dò kênh . . . 20
3.9 Giao thức và định dạng bản tin . . . 20
3.10 Đồng bộ kênh và cấu hình Wi-Fi . . . 21
3.11 Giao diện web . . . 21
3.11.1 Hướng dẫn các chức năng giao diện Web . . . 22

CHƯƠNG 4: KẾT QUẢ THỰC HIỆN VÀ KIỂM THỬ **26**
4.1 Kết quả thực nghiệm và kiểm thử . . . 26
4.2 Phương pháp kiểm thử . . . 27
4.3 Kịch bản kiểm thử và kết quả [7] . . . 27
4.4 Các vấn đề gặp phải và cách xử lý . . . 30

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **31**
5.1 Kết quả đạt được . . . 31
5.2 Hạn chế . . . 31
5.3 Hướng phát triển . . . 32
5.4 Kết luận . . . 32

TÀI LIỆU THAM KHẢO **vii**

A MÃ NGUỒN **viii**
A.1 Danh mục tệp mã nguồn . . . viii
A.2 Khai báo gói tin ESP-NOW (MCU alexa, ESP-IDF) . . . viii

A.3 Gửi lệnh ưu tiên ESP-NOW, dự phòng MQTT ix
A.4 Tác vụ nhận dạng giọng nói x
A.5 Điều khiển tốc độ quạt bằng PWM xi
A.6 Tải tệp ghi âm lên máy chủ AI xi
A.7 Hàng đợi đọc câu trả lời xiii
A.8 Gửi lệnh từ giao diện web xiv

DANH MỤC HÌNH VẼ

3.1.1: Yêu cầu chức năng 9
3.1.2: Yêu cầu phi chức năng 10
3.2: Kiến trúc tổng thể của hệ thống 10
3.5: Máy trạng thái xử lý giọng nói trên MCU alexa 14
3.6: Chuỗi hỏi–đáp từ lệnh bắt đầu đến khi loa đọc câu trả lời 16
3.8.1: Hành vi của hai cảm ứng TTP223B 19
3.11: Giao diện web điều khiển và giám sát 22
3.11.1.1: Giao diện đổi MQTT 23
3.11.1.2: Giao diện web điều khiển và giám sát (Light Mode). 23
3.11.1.3: Giao diện thêm phòng mới. 24
3.11.1.4: Giao diện thêm thiết bị mới 24
4.1 MCU center (ESP32 DevKit V1) với cảm biến DHT11 và nút nhấn 26
4.2 Hai ESP32-S3: MCU alexa và MCU recording kèm micro, thẻ SD, module âm thanh và loa. 26
4.3 Trang web cấu hình Wi-Fi và MQTT (192.168.4.1). 29
4.4 MCU control với hai cảm ứng, LEDRGB, MOSFET và quạt. 29

DANH MỤC BẢNG

1.2 Đánh giá so sánh các sản phẩm và nền tảng thông minh tương tự thực tế 1
1.4: Phạm vi của đề tài 2
1.6: Sơ lược bố cục và nội dung báo cáo 3
3.1.1: Yêu cầu chức năng 9
3.1.2: Yêu cầu phi chức năng 10
3.2: Vai trò của bốn vi điều khiển 11
3.3.1: Danh mục linh kiện chính 11
3.3.2: Gán chân GPIO theo từng vi điều khiển 12
3.4: Phần mềm trên từng vi điều khiển 13
3.5.2.1: Lệnh MultiNet và bản tin tương ứng 14
3.5.2.2: Màu LED trạng thái trên MCU alexa 15
3.6.2: Tham số ghi âm 16
3.6.5: Bản tin UART và phản ứng của MCU recording 17
3.7.1: Bản tin vào MCU center và hành động 18
3.7.3: Các đường dẫn của web server cấu hình 18
3.8.1: Hành vi của hai cảm ứng TTP223B 19
3.8.3: Mức quạt và giá trị PWM 19
3.9.1: Gói tin ESP-NOW 20
3.9.2: Topic MQTT 20
3.9.3: Dữ liệu trên đường UART giữa MCU alexa và MCU recording 21
4.1 Đối chiếu kết quả với mục tiêu 27
4.2 Kịch bản kiểm thử chức năng 27
4.3 Độ trễ từ lúc dứt lệnh đến lúc thiết bị đổi trạng thái 30
4.4 Tỷ lệ nhận dạng lệnh theo khoảng cách 30
4.5 Vấn đề gặp phải, nguyên nhân và giải pháp 30
5.1 Hạn chế hiện tại và hướng khắc phục 31
A.1 Danh mục tệp mã nguồn của đề tài viii

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa đầy đủ
AP	Access Point (Điểm truy cập không dây)
DC	Direct Current (Dòng điện một chiều)
DHT	Digital Humidity and Temperature (Cảm biến nhiệt độ, độ ẩm)
ESP-NOW	Giao thức truyền thông không dây công suất thấp của Espressif
INMP	Integrated Micro-Electro-Mechanical Systems Microphone
LED	Light Emitting Diode (Đèn đi-ốt phát quang)
MCU	Microcontroller Unit (Vi điều khiển)
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
MQTT	Message Queuing Telemetry Transport
NVS	Non-Volatile Storage (Bộ nhớ lưu trữ không mất dữ liệu)
PCB	Printed Circuit Board (Bo mạch in)
PWA	Progressive Web App
RGB	Red Green Blue (Hệ màu Đỏ - Xanh lá - Xanh dương)
SD	Secure Digital (Thẻ nhớ)
TLS	Transport Layer Security
TTS	Text-To-Speech (Chuyển đổi văn bản thành giọng nói)
UART	Universal Asynchronous Receiver-Transmitter
Wi-Fi	Wireless Fidelity

CHƯƠNG 1: GIỚI THIỆU

1.1 Lý do chọn đề tài

– Bối cảnh: IoT và nhà thông minh đang phát triển; khối xử lý trung tâm là "bộ não"điều phối tương tác giữa người dùng và các thiết bị ngoại vi.

– Nhu cầu: bộ trung tâm hiện đại không chỉ điều khiển từ xa qua Internet/app mà còn phải linh hoạt về giao diện tương tác, trong đó giọng nói là cách tự nhiên nhất.

– Vấn đề thực tế: nhiều trợ lý giọng nói phụ thuộc hoàn toàn vào đám mây (mất mạng là mất điều khiển, độ trễ cao với lệnh đơn giản như bật đèn). Đề tài đặt câu hỏi: có thể tách lệnh điều khiển (xử lý cục bộ, offline) khỏi hỏi đáp mở (cần AI đám mây) trên vi điều khiển giá thấp hay không.

– Ý nghĩa học tập: tích hợp nhận dạng giọng nói, truyền thông không dây, điều khiển công suất và giao diện web trên một hệ thống hoàn chỉnh.

1.2 Sản phẩm tương tự thực tế

Bảng 1.2: Đánh giá so sánh các sản phẩm và nền tảng thông minh tương tự thực tế

Sản phẩm / nền tảng	**Điểm nổi bật (nhận định chung)**	**Hạn chế so với mục tiêu đề tài**
Amazon Alexa / Echo	Nhận dạng từ khóa tốt, hệ sinh thái thiết bị rộng	Xử lý chủ yếu trên đám mây; không hỗ trợ tiếng Việt chính thức
Google Home / Assistant	Hỗ trợ tiếng Việt, nhiều thiết bị tương thích	Phụ thuộc tài khoản và đám mây, khó tùy biến phần cứng
Xiaomi Mi Home	Giá dễ tiếp cận, nhiều thiết bị	Hệ sinh thái riêng, ít linh hoạt khi tự chế thiết bị
Giải pháp trong nước (Lumi, BKAV SmartHome...)	Phù hợp thị trường Việt Nam, có hỗ trợ kỹ thuật	Chi phí triển khai cao hơn mô hình tự xây dựng; ít mở cho sinh viên nghiên cứu
Home Assistant + ESP32 (mã nguồn mở)	Rất linh hoạt, cộng đồng lớn	Cần máy chủ riêng; cấu hình phức tạp với người mới

– Nhận xét: các giải pháp thương mại mạnh về hoàn thiện nhưng thường phụ thuộc đám mây, khó can thiệp phần cứng. Đề tài đi theo hướng tự chủ phần cứng + lệnh offline + AI đám mây chỉ khi cần.

1.3 Mục tiêu của đề tài

Mục tiêu chung: Thiết kế và thi công bộ điều khiển trung tâm có khả năng kết nối, quản lý và điều khiển tập trung các thiết bị trong nhà thông minh, cho phép tương tác trực tiếp qua giọng nói hoặc giao diện ứng dụng di động/web.

Mục tiêu cụ thể:

1. **Xử lý giọng nói hai tầng:**
   - *Tầng 1 (Offline trên chip):* Nhận diện từ khóa kích hoạt và thực thi trực tiếp các lệnh điều khiển thiết bị cơ bản (đèn, quạt).
   - *Tầng 2 (Cloud AI):* Kết nối đám mây để hỗ trợ hỏi đáp thông minh thông qua Gemini AI.
2. **Phản hồi âm thanh TTS:** Tiếp nhận chuỗi văn bản, quản lý thông qua hàng đợi an toàn đa luồng và phát âm thanh ra loa thông qua chuẩn giao tiếp I2S.
3. **Truyền thông không dây linh hoạt:**
   - *ESP-NOW:* Kênh ưu tiên nội bộ, truyền thông trực tiếp giữa các thiết bị mà không cần router Wi-Fi.
   - *MQTT:* Kênh đồng bộ đám mây và đóng vai trò dự phòng (fallback) khi cần thiết.
4. **Thi công mô hình đại diện:**
   - **Đèn RGB:** Đảm nhận chức năng bật/tắt và đổi màu linh hoạt.
   - **Quạt DC:** Điều khiển qua MOSFET AOD4184 với tải điều chỉnh công suất sử dụng xung PWM.

   Việc tích hợp hai dạng tải này minh chứng cho khả năng mở rộng của bộ trung tâm sang các hệ thống thiết bị thực tế khác.
5. **Giao diện web:** Xây dựng *Web App* phục vụ giám sát nhiệt độ, độ ẩm cùng tính năng điều khiển thiết bị, kết hợp với trang *Web Config* hỗ trợ nạp cấu hình WiFi và MQTT thuận tiện.

1.4 Đối tượng, phạm vi và giới hạn

Bảng 1.4: Phạm vi của đề tài

Nội dung	**Trong phạm vi**	**Ngoài phạm vi**
Ngôn ngữ	Lệnh thoại tiếng Anh; hỏi đáp và phản hồi tiếng Việt	Lệnh thoại tiếng Việt ngoại tuyến
Thiết bị	Đèn LED RGB, quạt DC, cảm biến DHT11	Tải điện lưới, điều khiển rèm cửa
Mạng	Mạng cục bộ (Wi-Fi, ESP-NOW, MQTT broker nội bộ)	Truy cập từ Internet bên ngoài mạng nhà
Ứng dụng	Web app chạy trên trình duyệt, kết nối MQTT	Ứng dụng di động đóng gói, cơ sở dữ liệu
Mức hoàn thiện	Mô hình thử nghiệm trên breadboard	Sản phẩm thương mại, PCB, đạt chuẩn an toàn điện

1.5 Phương pháp thực hiện

Đề tài đi theo quy trình thiết kế hệ thống nhúng có định hướng yêu cầu: xác định yêu cầu và kịch bản sử dụng, thiết kế phần cứng (sơ đồ khối, danh mục linh kiện), thiết kế phần mềm (module, máy trạng thái), cài đặt từng bước từ giao tiếp, cảm biến, cơ cấu chấp hành đến hành vi tổng thể, rồi kiểm tra theo hai mức: verification (hệ thống có đúng đặc tả không) và validation (hệ thống có đáp ứng nhu cầu người dùng không). Mỗi phiên bản firmware được đối chiếu với nhật ký Serial và thử trên phần cứng thật.

1.6 Sơ lược bố cục báo cáo

Bảng 1.6: Sơ lược bố cục và nội dung báo cáo

Chương	**Nội dung**
1. Giới thiệu	Lý do, sản phẩm tương tự, mục tiêu, bố cục
2. Cơ sở lý thuyết	IoT, linh kiện, giao thức và chuẩn giao tiếp, nhận dạng giọng nói
3. Thiết kế hệ thống	Yêu cầu, phần cứng, phần mềm, lưu đồ
4. Kết quả và kiểm thử	Mô hình thực tế, giao diện, kiểm thử, nhận xét
5. Kết luận	Kết quả, hạn chế, hướng phát triển

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về IoT và nhà thông minh

2.1.1 Giới thiệu về Internet of Things (IoT)

Internet of Things (IoT) là hệ thống kết nối các thiết bị vật lý, cảm biến, bộ truyền động và các công nghệ khác thông qua mạng Internet nhằm thu thập, truyền tải và xử lý dữ liệu tự động mà không đòi hỏi sự can thiệp trực tiếp từ con người. Một hệ thống IoT tiêu chuẩn cấu thành từ các thành phần cốt lõi sau:

- **Thiết bị (Devices/Sensors):** Các cảm biến thu thập thông tin từ môi trường hoặc thiết bị chấp hành thực thi hành động.
- **Kết nối (Connectivity):** Hạ tầng mạng truyền dẫn dữ liệu (Wi-Fi, Zigbee, ESP-NOW...) kết nối thiết bị với đám mây hoặc các nút mạng khác.
- **Nền tảng đám mây (Cloud Platform):** Nơi lưu trữ, quản lý và xử lý dữ liệu lớn (Big Data).
- **Ứng dụng (Application):** Giao diện người dùng cuối (Web App, Mobile App) để hiển thị trực quan thông tin và điều khiển hệ thống từ xa.

Về mặt kiến trúc, hệ thống IoT được phân chia thành 4 lớp chức năng chính:

1. **Lớp cảm nhận (Perception Layer):** Thu thập dữ liệu vật lý thông qua các cảm biến.
2. **Lớp mạng (Network Layer):** Truyền tải dữ liệu thông qua các giao thức truyền thông không dây hoặc có dây.
3. **Lớp xử lý (Processing Layer):** Lưu trữ, phân tích và xử lý dữ liệu trên biên (edge) hoặc đám mây (cloud).
4. **Lớp ứng dụng (Application Layer):** Tương tác trực tiếp với người dùng qua giao diện quản lý.

Trong đề tài, tổng quan về IoT dùng để làm nền tảng kiến trúc kết nối các thiết bị phần cứng nhúng với giao diện web quản lý và hệ thống đám mây.

2.1.2 Mô hình nhà thông minh và phân tầng xử lý

Mô hình nhà thông minh (Smart Home) hiện đại là một hệ sinh thái tự động hóa bao gồm:

- **Bộ điều khiển trung tâm (Hub):** Đóng vai trò "bộ não"thu thập dữ liệu, điều phối và ra quyết định.
- **Thiết bị cảm biến (Sensors):** Thu thập thông số môi trường (nhiệt độ, độ ẩm, âm thanh...).
- **Thiết bị chấp hành (Actuators):** Thực thi lệnh điều khiển vật lý (đèn, quạt, van nước...).

– **Giao diện người dùng (User Interfaces):** Nút bấm, màn hình LCD, Web App để giám sát và điều khiển.

Trong kiến trúc điều khiển thông minh, việc phân biệt giữa **xử lý cục bộ (Edge/Offline Processing)** và **xử lý đám mây (Cloud Processing)** đóng vai trò cốt lõi:

– *Xử lý cục bộ (Edge):* Thực hiện ngay trên vi điều khiển trung tâm tại chỗ, đảm bảo tốc độ phản hồi tức thì, độ tin cậy cao và hoạt động độc lập không cần kết nối Internet.

– *Xử lý đám mây (Cloud):* Tận dụng năng lực tính toán của AI máy chủ từ xa để giải quyết các tác vụ phức tạp như xử lý ngôn ngữ tự nhiên và hỏi đáp mở.

Trong đề tài, mô hình này được ứng dụng trực tiếp để xây dựng kiến trúc xử lý giọng nói hai tầng (tầng 1 offline cục bộ và tầng 2 cloud AI).

2.2 Vi điều khiển và linh kiện sử dụng

2.2.1 ESP32-S3 (MCU alexa và MCU recording) [1]

- **Nguyên lý và thông số chính:** SoC hai lõi Xtensa LX7 của Espressif, tích hợp Wi-Fi 2.4 GHz và Bluetooth LE. Chip được trang bị tập lệnh vector hỗ trợ tính toán mạng nơ-ron, hỗ trợ PSRAM và các giao tiếp ngoại vi như I2S, SPI, UART, rất phù hợp cho các bài toán nhận dạng giọng nói và xử lý âm thanh thời gian thực.
- **Ứng dụng trong đề tài:** Dùng cho hai vi điều khiển thành phần:
  - **MCU recording:** Ghi âm từ micro INMP441 qua giao tiếp I2S, lưu thành tệp WAV (16 kHz, 16 bit) trên thẻ SD và tải lên máy chủ AI qua giao thức HTTP. MCU này nhận câu trả lời từ MQTT client, sau đó phát ra loa qua DAC MAX98357A bằng công nghệ TTS tiếng Việt.
  - **MCU alexa:** Chạy thư viện ESP-SR để nhận dạng từ khóa "alexa" và lệnh tiếng Anh trực tiếp ngay trên chip mà không cần kết nối đám mây [8].
- **Cấu hình truyền thông:** Cả hai vi điều khiển đóng vai trò là MQTT client. Broker MQTT được triển khai bên ngoài, không chạy trực tiếp trên các chip ESP32-S3 này.

Trong đề tài, ESP32-S3 chạy ESP-IDF dùng để thực hiện thuật toán xử lý giọng nói offline, nhận diện từ khóa đánh thức (Wakeword) và điều phối lệnh trung tâm.

2.2.2 ESP32 DevKit V1 (MCU center) [1]

– **Nguyên lý & Thông số chính:** Sử dụng chip ESP32-WROOM-32, tích hợp Wi-Fi và Bluetooth, nhiều chân GPIO.

Trong đề tài, chỉ làm đầu mối nhận và gửi lệnh điều khiển cho giao diện người dùng.

- Làm cầu nối hai chiều giữa ESP-NOW và MQTT, để web app điều khiển được MCU control.
- Đọc cảm biến DHT11 mỗi 6 giây và đăng dữ liệu lên MQTT.
- Phát điểm truy cập (AP) TDTHINH-xxxx với trang web cấu hình Wi-Fi/MQTT, và phát gói cấu hình Wi-Fi cho các MCU khác bằng nút GPIO15.

2.2.3 ESP32 28 chân (MCU control) [1]

– **Nguyên lý & Thông số chính:** Vi điều khiển ESP32 nhỏ gọn, hỗ trợ giao tiếp không dây ESP-NOW và PWM xuất xung điều khiển công suất.

Trong đề tài, thực thi lệnh: PWM quạt, đèn RGB, cảm ứng, lưu NVS.

2.2.4 Micro INMP441

– **Nguyên lý & Thông số chính:** Micro MEMS kỹ thuật số đầu ra chuẩn I2S, độ nhạy cao, tỉ số tín hiệu trên nhiễu (SNR) tốt, hoạt động ở điện áp 3.3V.

Trong đề tài, Micro INMP441 dùng để thu nhận tín hiệu âm thanh giọng nói của người dùng.

2.2.5 Mạch khuếch đại MAX98357A và Loa [2]

– **Nguyên lý & Thông số chính:** Mạch khuếch đại âm thanh Class-D kỹ thuật số nhận tín hiệu trực tiếp qua chuẩn I2S, không cần DAC ngoài, công suất đầu ra khoảng 3W ở 4Ω.

Trong đề tài, mạch MAX98357A dùng để khuếch đại tín hiệu âm thanh và phát phản hồi TTS ra loa.

2.2.6 Module thẻ nhớ SD và thẻ SD 4GB

– **Nguyên lý & Thông số chính:** Module giao tiếp thẻ nhớ SD qua chuẩn SPI, hỗ trợ định dạng FAT16/FAT32, dung lượng lưu trữ lớn.

Trong đề tài, module thẻ SD dùng để lưu tạm file ghi âm `rec.wav` *nhằm tiết kiệm bộ nhớ RAM của vi điều khiển.*

2.2.7 Cảm biến nhiệt độ, độ ẩm DHT11 [3]

– **Nguyên lý & Thông số chính:** Cảm biến đo nhiệt độ và độ ẩm môi trường sử dụng giao tiếp 1 dây (Single-wire digital interface), dải đo nhiệt độ từ0–50°C (±2°C)à độ ẩm 20–90% (±5%).

Trong đề tài, cảm biến DHT11 dùng để đo đạc các thông số môi trường phòng và cập nhật lên hệ thống Web Dashboard.

2.2.8 Nút cảm ứng điện dung TTP223B [4]

– **Nguyên lý & Thông số chính:** IC cảm ứng điện dung đơn kênh, thay thế nút nhấn cơ học truyền thống, xuất tín hiệu số (Digital output) khi có chạm.

Trong đề tài, nút TTP223B dùng làm phím cảm ứng để bật/tắt thiết bị thủ công hoặc kích hoạt chế độ cấu hình.

2.2.9 MOSFET AOD4184 và Động cơ DC [5]

– **Nguyên lý & Thông số chính:** MOSFET kênh N AOD4184A (40 V, vỏ TO-252) có điện áp ngưỡng $V_{GS(th)}$ từ 1,7 V đến 2,6 V (điển hình 2,1 V). Điện áp 3,3 V từ ESP32 cao hơn ngưỡng tối đa khoảng 0,7 V nên MOSFET dẫn được nhưng chưa ở trạng thái mở hoàn toàn; datasheet chỉ nêu $R_{DS(on)}$ tại $V_{GS} = 4{,}5$ V (tối đa 9,5 mΩ) và $V_{GS} = 10$ V (tối đa 7 mΩ). Dòng liên tục tối đa là 50 A tại $T_C = 25°C$ (40 A tại 100°C) [5]. Với quạt DC công suất nhỏ của đề tài, dòng tải thấp hơn nhiều so với giới hạn này.

Trong đề tài, MOSFET AOD4184 dùng để đóng cắt và điều khiển công suất quạt DC theo tín hiệu xung PWM.

2.2.10 Đèn LED RGB (HW-179)

– **Nguyên lý & Thông số chính:** Mô-đun đèn LED tích hợp 3 màu cơ bản (Red, Green, Blue), điều khiển độ sáng từng kênh thông qua xung PWM 3 kênh độc lập.

Trong đề tài, đèn LED RGB dùng làm tải đại diện cho nhóm thiết bị bật/tắt và đổi màu trạng thái.

2.3 Chuẩn giao tiếp và giao thức truyền thông

Trong hệ thống nhúng phân tán, việc kết hợp các chuẩn giao tiếp phần cứng và giao thức mạng đóng vai trò quyết định độ ổn định:

– **ESP-NOW:** Giao thức truyền thông không dây P2P của Espressif, hoạt động trực tiếp qua khung MAC Wi-Fi ở băng tần 2.4GHz với tốc độ mặc định 1 Mbps (tối đa 241 byte/gói), không cần qua Router. *Trong đề tài, ESP-NOW dùng làm kênh truyền thông nội bộ tốc độ cao giữa các vi điều khiển. [9],[10]*

– **MQTT:** Giao thức nhắn tin Publish/Subscribe tối ưu cho IoT, hỗ trợ 3 mức QoS (0, 1, 2). *Trong đề tài, MQTT dùng làm kênh đồng bộ dữ liệu với Web App và dự phòng khi ESP-NOW gặp sự cố [11].*

– **WebSocket:** Giao thức cung cấp kênh giao tiếp song công toàn phần (Full-duplex) qua một kết nối TCP duy nhất. *Trong đề tài, WebSocket dùng để trình duyệt web kết nối nhận thông tin trạng thái thời gian thực.*

– **HTTP / REST API:** Mô hình Client-Server truyền thống với phương thức POST multipart. *Trong đề tài, HTTP dùng để upload file ghi âm* `rec.wav` *lên Cloud AI và cấu hình Web Server trên ESP32.*

– **I2C (Inter-Integrated Circuit):** Bus nối tiếp đồng bộ 2 dây gồm đường dữ liệu (SDA) và xung nhịp (SCL), theo mô hình chủ–tớ; mỗi thiết bị có một địa chỉ riêng nên nhiều thiết bị dùng chung một cặp dây (cần điện trở kéo lên, tốc độ phổ biến 100 kb/s và 400 kb/s). *Trong đề tài, I2C không được sử dụng; các ngoại vi âm thanh dùng I2S, thẻ nhớ dùng SPI và cảm biến DHT11 dùng giao thức một dây riêng.*

- **I2S (Inter-IC Sound):** Chuẩn giao tiếp âm thanh số gồm 3 đường: xung bit (BCLK), chọn kênh (WS/LRC) và dữ liệu (SD). *Trong đề tài, I2S dùng để kết nối Micro INMP441 và mạch khuếch đại MAX98357A.*

- **SPI (Serial Peripheral Interface):** Giao tiếp đồng bộ 4 dây (SCK, MOSI, MISO, CS) tốc độ cao. *Trong đề tài, SPI dùng để giao tiếp với module thẻ nhớ SD (cấu hình tốc độ 20 MHz).*

- **UART:** Giao tiếp nối tiếp không đồng bộ 2 dây (TX/RX) ở tốc độ 115200 baud. *Trong đề tài, UART dùng để truyền thông giữa các MCU thành phần.*

- **PWM (Pulse Width Modulation):** Điều chế độ rộng xung với tần số 1 kHz cho quạt và 5 kHz cho LED RGB (độ phân giải 8-bit). *Trong đề tài, PWM dùng để điều chỉnh tốc độ quạt và độ sáng màu sắc đèn.*

Lưu ý kỹ thuật: Do ESP-NOW yêu cầu các thiết bị cùng kênh Wi-Fi, khi một MCU kết nối Router thì kênh của Router sẽ quyết định kênh ESP-NOW. Do đó hệ thống tích hợp cơ chế đồng bộ kênh tự động để đảm bảo tính ổn định. *Trong đề tài, cơ chế này dùng để duy trì liên kết không dây bền bỉ giữa các nút mạng.*

2.4 Nhận dạng giọng nói và AI

- **ESP-SR:** Bộ thư viện xử lý giọng nói nhúng của Espressif gồm 3 thành phần chính: *AFE* (lọc nhiễu âm thanh), *WakeNet* (phát hiện từ khóa đánh thức), và *MultiNet* (khớp lệnh thoại đã đăng ký theo xác suất, hỗ trợ tối đa 200 lệnh tiếng Anh ở định dạng 16 kHz, 16-bit mono). *Trong đề tài, ESP-SR dùng để nhận diện từ khóa "Alexa"và các lệnh điều khiển thiết bị offline [8], [12].*

- **Gemini API (REST):** Mô hình ngôn ngữ lớn (LLM) đám mây của Google. *Trong đề tài, Gemini API dùng để phân tích ngữ nghĩa các câu hỏi mở của người dùng [13].*

- **TTS (Text-to-Speech):** Công nghệ chuyển đổi văn bản thành âm thanh. *Trong đề tài, hệ thống sử dụng thư viện ESP32-audioI2S gọi Google TTS tiếng Việt, kết hợp cắt câu* ≤ 150 *byte để phát phản hồi qua loa. [6]*

Pipeline xử lý giọng nói: "Lọc – Đánh thức – Hiểu lệnh" (AFE → WakeNet → MultiNet). *Trong đề tài, pipeline này dùng để xử lý các câu lệnh giọng nói của người dùng: AFE lọc nhiễu, WakeNet phát hiện từ khóa "alexa", MultiNet nhận dạng lệnh. Mức độ hiệu quả được đánh giá ở Mục 4.3 (Bảng 4.4).*

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1 Phân tích yêu cầu

– Yêu cầu được rút ra từ các mục tiêu ở mục 1.3 và viết dưới dạng kiểm tra được, mỗi yêu cầu có tiêu chí để đối chiếu ở Chương 4.

Bảng 3.1.1: Yêu cầu chức năng

Mã	Yêu cầu	Tiêu chí kiểm tra
FR1	Nhận dạng từ khóa “Alexa” ngoại tuyến	Nói từ khóa “Alexa”, hệ thống phát hiện và phát âm thanh phản hồi.
FR2	Nhận dạng các lệnh điều khiển tiếng Anh ngoại tuyến	Lệnh được nhận dạng đúng và gửi đến bộ điều khiển trung tâm.
FR3	Hỏi đáp mở bằng tiếng Việt	Người dùng đặt câu hỏi, hệ thống trả lời bằng giọng nói.
FR4	Điều khiển đèn RGB	Bật/tắt, thay đổi màu sắc và cập nhật trạng thái đèn.
FR5	Điều khiển quạt DC	Thay đổi mức tốc độ quạt theo lệnh của người dùng.
FR6	Điều khiển bằng cảm ứng	Chạm cảm ứng để bật/tắt đèn hoặc quạt.
FR7	Giám sát nhiệt độ và độ ẩm	Đọc dữ liệu từ cảm biến DHT11 và gửi lên hệ thống.
FR8	Điều khiển qua giao diện Web	Người dùng có thể giám sát và điều khiển thiết bị từ trình duyệt.
FR9	Cấu hình Wi-Fi và MQTT	Cho phép thay đổi thông số cấu hình mà không cần nạp lại firmware.
FR10	Điều khiển khi mất kết nối ESP-NOW	Khi ESP-NOW không khả dụng, hệ thống có khả năng chuyển sang MQTT để gửi lệnh điều khiển thiết bị, với điều kiện kết nối MQTT và thiết bị đích vẫn hoạt động.
FR11	Điều khiển khi mất kết nối MQTT	Khi MQTT không khả dụng, hệ thống có khả năng sử dụng ESP-NOW để gửi lệnh điều khiển thiết bị, với điều kiện kết nối ESP-NOW và thiết bị đích vẫn hoạt động.

Bảng 3.1.2: Yêu cầu phi chức năng

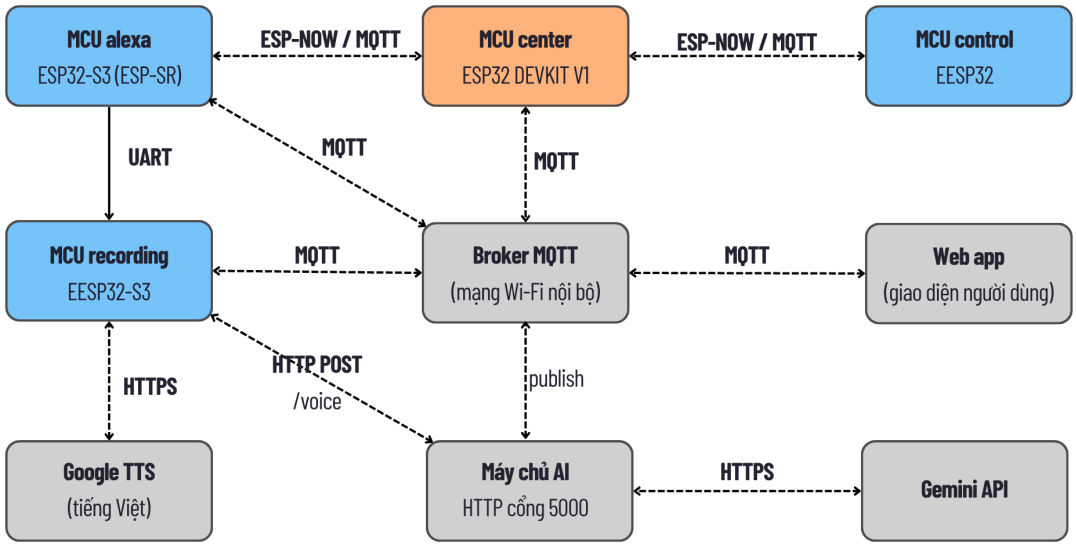
Mã	Yêu cầu	Tiêu chí kiểm tra
NFR1	Lệnh điều khiển không phụ thuộc Internet	Ngắt đường ra Internet, lệnh đèn và quạt vẫn thực hiện được.
NFR2	Độ trễ lệnh thấp	Đo từ lúc dứt lệnh đến lúc thiết bị đổi trạng thái; chỉ tiêu đặt sau khi đo ở mục 4.3.
NFR3	Tự phục hồi	Mất Wi-Fi thì tự nối lại; cấu hình giữ nguyên sau khi mất điện nhờ NVS.
NFR4	Dễ mở rộng	Thêm thiết bị bằng cách thêm một lệnh MultiNet và một chuỗi lệnh/topic, không đổi kiến trúc.
NFR5	An toàn khi thử nghiệm	Chỉ dùng tải điện áp thấp, không nối điện lưới.

3.2 Kiến trúc tổng thể

Hệ thống chia thành ba nhóm: giao diện giọng nói (MCU alexa và MCU recording), bộ điều phối trung tâm (MCU center) và bộ chấp hành (MCU control). Hình 3.2 cho thấy các khối và giao thức giữa chúng.

Ba nguyên tắc thiết kế chi phối kiến trúc:

1. **Tách vai trò theo chip.** Nhận dạng giọng nói và ghi/phát âm thanh chạy trên hai chip ESP32-S3 riêng, nên vòng lặp âm thanh không bị mạng làm gián đoạn.



Hình 3.2: Kiến trúc tổng thể của hệ thống

2. **Hai tầng xử lý giọng nói.** Lệnh cố định được nhận dạng và chuyển đi ngay trên chip; chỉ câu hỏi mở mới được ghi âm và gửi lên đám mây.
3. **Hai kênh truyền.** Lệnh điều khiển đi bằng ESP-NOW trước; MQTT phục vụ giao diện web và làm kênh dự phòng.

3.2: Vai trò của bốn vi điều khiển

MCU	Chip	Môi trường	Ngoại vi	Nhiệm vụ
MCU alexa	ESP32-S3	ESP-IDF v4.4.8, ESP-SR	Micro INMP441, LED WS2812, nút GPIO0	Nhận dạng "alexa" và lệnh; phát lệnh qua ESP-NOW/MQTT; điều phối ghi âm qua UART
MCU recording	ESP32-S3	Arduino-ESP32	Micro INMP441, MAX98357A và loa, module SD	Ghi WAV, tải lên máy chủ AI, nhận câu trả lời qua MQTT, đọc bằng TTS tiếng Việt
MCU center	ESP32 DevKit V1	Arduino-ESP32	DHT11, nút GPIO15	Cầu nối ESP-NOW và MQTT, dịch lệnh, đọc DHT11, trang web cấu hình
MCU control	ESP32 (28 chân)	Arduino-ESP32	2 cảm ứng TTP223B, LED RGB HW-179, MOSFET AOD4184 và quạt	Thực thi lệnh, xử lý cảm ứng, lưu màu và kênh

3.3 Thiết kế phần cứng

3.3.1 Danh mục linh kiện

Bảng 3.3.1: Danh mục linh kiện chính

Linh kiện	Số lượng	Giá tiền (VNĐ)	Dùng cho
ESP32-S3	2	390.000	MCU alexa, MCU recording
ESP32 DevKit V1	1	100.000	MCU center
ESP32 module 28 chân	1	75.000	MCU control
Micro MEMS INMP441	2	116.000	Mỗi chip ESP32-S3 một micro
Module MAX98357A và loa	1	55.000	Phát TTS
Module thẻ nhớ SD và thẻ 4 GB	1	68.000	Lưu tệp rec.wav tạm
Cảm biến DHT11	1	10.000	Đo nhiệt độ, độ ẩm
Cảm ứng TTP223B	2	4.000	Đèn và quạt
Module LED RGB HW-179	1	15.000	Đèn đại diện
Module MOSFET AOD4184 và động cơ DC	1	26.000	Quạt đại diện
Nút nhấn	1	500	Phát/xóa cấu hình trên MCU center
Module nguồn breadboard	1	32.000	Cấp nguồn 5V

3.3.2 Sơ đồ chân

Bảng 3.3.2: Gán chân GPIO theo từng vi điều khiển

MCU	Chức năng	GPIO	Ghi chú
alexa	LED trạng thái WS2812 (2 LED)	48	Điều khiển bằng RMT
alexa	Nút quét kênh/nhận cấu hình	0	Kéo lên, ngắt cạnh xuống
alexa	UART1 TX (tới recording RX)	17	115 200 baud, 8N1
alexa	UART1 RX (từ recording TX)	18	
alexa	Micro INMP441	12 / 11 / 10	Do esp_board_init của board cấu hình
recording	Micro INMP441: SCK / WS / SD	12 / 11 / 10	I2S0 nhận, 16 kHz
recording	MAX98357A: BCLK / LRC / DIN	15 / 9 / 8	I2S1 phát
recording	Module SD: CS / MOSI / SCK / MISO	1 / 5 / 4 / 6	SPI 20 MHz
recording	UART2 TX / RX	18 / 17	Nối chéo TX / RX với alexa
center	DHT11 (dữ liệu)	13	
center	Nút phát/xóa cấu hình	15	Kéo lên, tích cực mức thấp
control	TTP223B đèn	4	Tích cực mức cao
control	TTP223B quạt	15	Tích cực mức cao
control	MOSFET quạt	22	PWM 1 kHz, 8 bit
control	LED R / G / B	18 / 19 / 21	PWM 5 kHz, 8 bit
control	Nút BOOT (dò kênh)	0	Giữ từ 5 giây

3.3.3 Lưu ý thiết kế

Trong quá trình thiết kế và thi công phần cứng, cần tuân thủ các lưu ý kỹ thuật sau:

- **Giao tiếp UART:** Kết nối giữa MCU alexa và MCU recording phải nối chéo chân (`TX` của MCU này nối với `RX` của MCU kia) và bắt buộc phải dùng chung đường mass (`GND`).
- **Cấu hình Micro I2S:** Firmware ghi âm đọc dữ liệu trên kênh trái của chuẩn I2S, do đó chân `L/R` của cảm biến INMP441 phải được nối xuống `GND`.
- **Mạch điều khiển tải cảm (Quạt DC):** Quạt là tải cảm nên cần lắp thêm diode bảo vệ (flyback diode) mắc song song ngược với động cơ. Khuyến nghị cấp nguồn riêng cho quạt để tránh hiện tượng sụt áp khi khởi động làm reset vi điều khiển ESP32.
- **Mạng truyền thông:** Cả bốn vi điều khiển đều hoạt động trên cùng một kênh Wi-Fi / ESP-NOW, do đó nên đặt các thiết bị gần router Wi-Fi trong quá trình thử nghiệm để đảm bảo độ ổn định.

3.4 Tổng quan phần mềm

Mỗi vi điều khiển trong hệ thống chạy một firmware riêng biệt được tối ưu cho từng vai trò cụ thể. Toàn bộ bốn vi điều khiển đều được thiết kế kiến trúc phần mềm dựa trên mô hình **"cờ và hàng đợi"** (flags and queues).

Cụ thể, các hàm gọi lại (callback functions) của giao thức ESP-NOW hoặc MQTT chỉ thực hiện nhiệm vụ chép dữ liệu nhận được vào bộ đệm (buffer) hoặc hàng đợi (queue) và bật cờ thông báo, sau đó kết thúc ngay lập tức. Vòng lặp chính (`loop()` hoặc các task có ưu tiên phù hợp) mới đảm nhận việc đọc dữ liệu và xử lý logic. Mẫu thiết kế này đảm bảo các tác vụ xử lý không làm chặn (non-blocking) luồng hệ thống cũng như luồng truyền thông mạng Wi-Fi/ESP-NOW.

Bảng 3.4: Phần mềm trên từng vi điều khiển

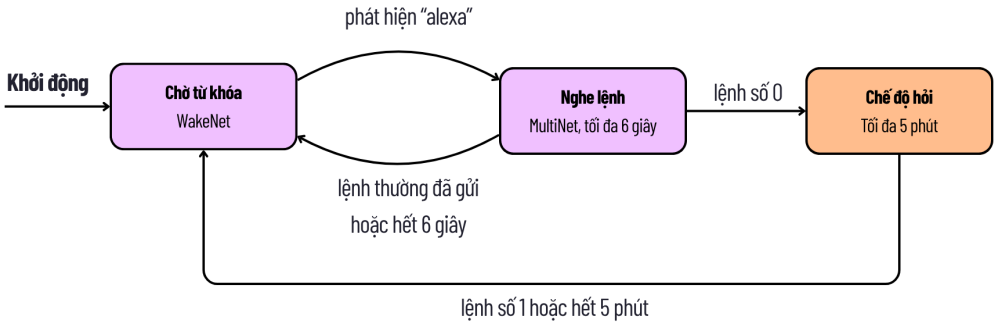
MCU	**Môi trường**	**Thư viện chính**	**Luồng chạy**
alexa	ESP-IDF v4.4.8	ESP-SR, esp_now, esp_mqtt, cJSON, led_strip	feed_Task (ưu tiên 10) và detect_Task (ưu tiên 5) trên lõi 1; uart_rx_task, net_worker, chan_hop, btn_task
recording	Arduino-ESP32	ESP32-audioI2S, SD, PubSubClient, ArduinoJson, Preferences	loop() và recordTask ghim lõi 1
center	Arduino-ESP32	PubSubClient, WebServer, DHT, ArduinoJson, Preferences	loop() và các cờ bất đồng bộ
control	Arduino-ESP32	esp_now, ArduinoJson, Preferences	loop() và hàng đợi ESP-NOW 20 phần tử [6]

3.5 Firmware MCU alexa

3.5.1 Luồng nhận dạng giọng nói

AFE được cấu hình một micro và một kênh tham chiếu, tắt khử vang (aec_init = false). Hai tác vụ chạy song song trên lõi 1: feed_Task đẩy dữ liệu I2S vào AFE, detect_Task lấy dữ liệu đã xử lý và chạy máy trạng thái trong Hình 3.5 [8].

Khi WakeNet phát hiện "alexa", MCU bật ESP-NOW, đặt LED trắng mờ, gửi bản tin "alexa" qua kênh ưu tiên và qua UART, rồi chuyển sang nghe lệnh. MultiNet chờ lệnh tối đa 6 000 ms. Lệnh số 0 đưa hệ thống vào chế độ hỏi (tối đa 5 phút, ESP-NOW tạm tắt để ưu tiên nghe); lệnh số 1 kết thúc câu hỏi và trả hệ thống về trạng thái chờ [12].



Hình 3.5: Máy trạng thái xử lý giọng nói trên MCU alexa

3.5.2 Danh sách lệnh

Bảng 3.5.2.1: Lệnh MultiNet và bản tin tương ứng

ID	Chuỗi âm vị trong mã	Ý nghĩa	Bản tin gửi đi	Hành động tại chỗ
0	haVcKWfSpcN	Bắt đầu hỏi	"haVcKWfSpcN"	LED đỏ, vào chế độ hỏi
1	fND KWfSpcN	Kết thúc hỏi	"fNDKWfSpcN"	LED xanh dương 1s, về chờ
2	TkN eF jc LiTS	Tắt các đèn	"TkN eF jc LiTS"	Tắt tất cả đèn
3	GRmN LiT	Đèn xanh lá	"GRmN LiT"	LED trạng thái xanh lá
4	RfD LiT	Đèn đỏ	"RfD LiT"	LED trạng thái đỏ
5	BLo LiT	Đèn xanh dương	"BLo LiT"	LED trạng thái xanh dương
6	WiT LiT	Đèn trắng	"WiT LiT"	Bật đèn màu trắng
7	TkN nN jc LiTS	Bật các đèn	"TkN nN jc LiTS"	Bật tất cả đèn
8	WiFm	Gửi cấu hình Wi-Fi	UART JSON	Gửi JSON {ssid, pass, mqtt, channel} qua UART
9	FaN eF	Tắt quạt	"fan_off"	Tắt động cơ quạt
10	FaN nN WcN	Bật quạt 25%	"fan_on_25"	Điều khiển PWM quạt 25%
11	FaN nN To	Bật quạt 50%	"fan_on_50"	Điều khiển PWM quạt 50%
12	FaN nN vRm	Bật quạt 75%	"fan_on_75"	Điều khiển PWM quạt 75%
13	FaN nN FeR	Bật quạt 100%	"fan_on_100"	Điều khiển PWM quạt 100%
14	TkN nN jc FaN	Bật quạt	"fan_on"	Bật quạt mức mặc định
15	TkN eF jc FaN	Tắt quạt	"fan_off"	Tắt quạt
16	YfLb LiT	Đèn vàng	"YfLb LiT"	Bật đèn màu vàng
17	eRcNq LiT	Đèn cam	"eRcNq LiT"	Bật đèn màu cam

(Tiếp theo Bảng 3.5.2.1)

ID	Chuỗi âm vị trong mã	Ý nghĩa	Bản tin gửi đi	Hành động tại chỗ
18	PglK LiT	Đèn hồng	“PglK LiT”	Bật đèn màu hồng
19	PkPcL LiT	Đèn tím	“PkPcL LiT”	Bật đèn màu tím

MultiNet nhận dạng lệnh theo cách phát âm chứ không đọc chữ viết, nên mỗi lệnh tiếng Anh được khai báo dưới dạng chuỗi ký hiệu âm vị theo quy ước của ESP-SR ; ví dụ “turn on the lights” ứng với “`TkN nN jc LiTS`“. Các chuỗi này được nạp từ cấu hình dự án (tệp `sdkconfig`). Khi nghe đúng một cụm lệnh, MultiNet chỉ trả về số thứ tự lệnh (ID), và chương trình gán ID đó thành bản tin tương ứng như trong Bảng 3.1. Cột ý nghĩa của bảng được suy ra từ phát âm của chuỗi âm vị và tên bản tin gửi đi.

Bảng 3.5.2.2: Màu LED trạng thái trên MCU alexa

Màu	Giá trị (R,G,B)	Trạng thái
Trắng mờ	80,80,80	Đã nghe “alexa”, chờ lệnh
Đỏ	255,0,0	Chế độ hỏi
Xanh dương	0,0,255	Vừa nhận lệnh kết thúc hỏi (1 giây)
Vàng nhấp nháy	255,255,0	Đang quét kênh tìm cấu hình
Tím	128,0,128	Đang nối Wi-Fi và MQTT sau khi nhận cấu hình
Hồng tím	255,0,255	Chưa có cấu hình trong NVS

3.5.3 Mạng và gửi lệnh

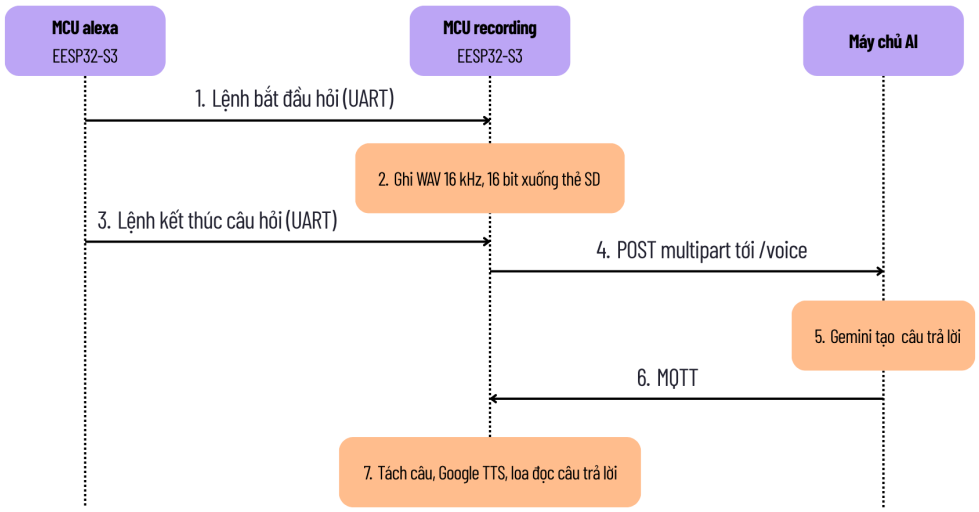
MCU alexa tắt chế độ tiết kiệm điện Wi-Fi (`WIFI_PS_NONE`) để giảm độ trễ ESP-NOW. Hàm gửi lệnh ưu tiên ESP-NOW: chờ trạng thái gửi trong tối đa 200 ms; nếu thất bại và MQTT đang kết nối, bản tin được xuất bản lên MQTT với QoS 1. Chức năng ESP-NOW chỉ được bật trong phiên tương tác sau khi nghe “alexa”, ngoài ra bị khóa để dành tài nguyên cho nhận dạng giọng nói.

3.6 Firmware MCU recording

3.6.1 Chuỗi hỏi–đáp

Hình 3.11 trình bày chuỗi xử lý một câu hỏi, từ lệnh bắt đầu đến khi loa đọc câu trả lời. File âm thanh được ghi thẳng xuống thẻ SD thay vì giữ trong RAM, nên độ dài câu hỏi không bị giới hạn bởi bộ nhớ của chip.

3.6.2 Ghi âm



Hình 3.6: Chuỗi hỏi–đáp từ lệnh bắt đầu đến khi loa đọc câu trả lời.

Bảng 3.6.2: Tham số ghi âm

Tham số	**Giá trị**
Tần số lấy mẫu	16 000 Hz
Khung I2S	32 bit, chuẩn Philips, đọc kênh trái
Tệp WAV	PCM 16 bit, một kênh, tiêu đề 44 byte
Dịch bit và khuếch đại	Dịch phải 14 bit, nhân 5,0, bão hòa về 16 bit
Bỏ đoạn đầu	300 ms
Kéo dài sau lệnh kết thúc	2 000 ms
Thời lượng tối đa	30 000 ms
Kích thước tối thiểu hợp lệ	9 600 byte (0,3 giây)
Ghi thẻ SD	Mỗi khi đủ hơn 768 mẫu
Tác vụ ghi âm	Ghim lõi 1, ưu tiên 5, ngăn xếp 6 144 byte

3.6.3 Tải lên máy chủ AI

Sau khi đóng tệp, firmware mở kết nối TCP tới máy chủ và gửi HTTP/1.1 POST dạng `multipart/form-data` tới đường dẫn `/voice` (cổng 5000), phần thân gồm tệp `rec.wav`. Firmware chờ phản hồi tối đa 10 giây. Câu trả lời thực sự đến qua MQTT (topic `esp32/gemini`) dưới dạng văn bản thuần hoặc JSON có khóa `answer`, nên phản hồi HTTP chỉ xác nhận đã nhận tệp.

3.6.4 Hàng đợi đọc câu trả lời [6]

Dịch vụ TTS giới hạn độ dài mỗi yêu cầu, nên văn bản trả lời được xử lý như sau:

1. Tách ở cuối câu (dấu chấm, chấm than, chấm hỏi, chấm phẩy, xuống dòng), không tách ở dấu thập phân như "3.5".
2. Câu dài hơn 150 byte được cắt tại khoảng trắng, không cắt giữa ký tự UTF-8; ngưỡng 150 byte thấp hơn giới hạn của dịch vụ vì tiếng Việt có dấu chiếm 2–3 byte mỗi ký tự.
3. Các câu ngắn được gộp thành một đoạn không quá 150 byte.
4. Hàng đợi chứa tối đa 12 đoạn; mỗi lượt `loop()` chỉ phát đoạn kế tiếp khi đoạn trước đã xong (sau tối thiểu 800 ms) hoặc đã quá 20 giây.
5. Việc phát bị hoãn trong lúc đang ghi âm.

3.6.5 Lệnh UART nhận từ MCU alexa

Bảng 3.6.5: Bản tin UART và phản ứng của MCU recording

Bản tin	**Phản ứng**
"alexa"	Đọc "Dạ, em nghe"
"haVcKWfSpcN"	Đọc "bạn hỏi đi", sau 500 ms bắt đầu ghi âm
"fNDKWfSpcN"	Hẹn dừng ghi sau 2 giây
"to_on" hoặc "ONRGB"	Chờ 400 ms; nếu không có màu cụ thể thì bật lại màu gần nhất và đọc "bật đèn"
"to_off" hoặc "OFFRGB"	Đọc "tắt đèn"
"R G B" hoặc JSON {r, g, b}	Lưu màu, đọc "Đã bật đèn"
"fan_off", "fan_on", "fan_on_25/50/75/100"	Đọc "tắt quạt", "bật quạt", "bật quạt mức N"
JSON {ssid, pass, mqtt}	Lưu NVS rồi khởi động lại

3.7 Firmware MCU center

3.7.1 Hoạt động chính

Vòng `loop()` lần lượt kiểm tra nút bấm, phục vụ web server, xử lý bản tin vừa nhận, kết nối lại Wi-Fi nếu rớt (quá 10 lần thất bại thì chuyển về chế độ AP), phát cấu hình khi được bật, duy trì MQTT (thử lại 5 giây một lần) và đọc DHT11 mỗi 6 giây. Bản tin nhận qua ESP-NOW hoặc MQTT đều đi vào cùng một hàm xử lý.

bảng 3.7.1: Bản tin vào MCU center và hành động

Bản tin	**Hành động**
“temp”, “dht”, “read_dht”	Đọc DHT11 ngay (cách lần đọc trước ít nhất 2 giây) và trả kết quả qua topic `esp32/send`
“TkN nNjc LiTS”, “to_on”	Gửi “ONRGB” tới topic `esp32/rgb` và qua ESP-NOW
“TkN eF jc LiTS”, “eF LiT”, “to_off”	Gửi “OFFRGB” tới topic `esp32/rgb` và qua ESP-NOW
“GRmN LiT”, “RfD LiT”, “BLo LiT”	Gọi hàm màu tương ứng; hiện là hàm rỗng, chưa phát lệnh màu
“fan_off”, “fan_on_N”, “fan_speed_N”	Chuẩn hóa thành “fan_off” hoặc “fan_on_N”, gửi tới topic `esp32/fan` và qua ESP-NOW
JSON {r,g,b}	Chuyển nguyên văn tới `esp32/rgb`, ưu tiên ESP-NOW
Bản tin khác	Phản hồi “ACK from ESP32 DevKit:” qua `esp32/send`

Có hai hàm gửi. Hàm gửi ưu tiên chỉ dùng ESP-NOW khi MQTT không sẵn sàng; hàm gửi song song luôn đẩy cả hai kênh để web app thấy trạng thái. Lệnh bật/tắt đèn, quạt và dữ liệu DHT11 dùng hàm gửi song song; JSON màu và phản hồi ACK dùng hàm gửi ưu tiên.

3.7.2 Đọc DHT11

Cứ 6 giây MCU đọc nhiệt độ và độ ẩm, tạo chuỗi dạng:

```
DHT11-> Temp: 30.8C | Hum: 72.9%
```

rồi gửi qua cả ESP-NOW và topic `esp32/dht`. Nếu đọc lỗi, MCU bỏ qua chu kỳ đó và thử lại ở chu kỳ sau.

3.7.3 Trang web cấu hình

Khi không nối được mạng đã lưu sau 10 giây, MCU center phát điểm truy cập có tên “TDTHINH-” cộng hai byte cuối của địa chỉ MAC AP (hệ thập lục phân), kênh 1, không mật khẩu. Người dùng mở trình duyệt tới địa chỉ `192.168.4.1` để chọn Wi-Fi, nhập mật khẩu và địa chỉ MQTT broker.

Bảng 3.7.3: Các đường dẫn của web server cấu hình

Đường dẫn	**Chức năng**
/	Trang cấu hình (HTML)
/getConfig	Trả cấu hình hiện có dạng JSON
/scanWifi	Quét mạng Wi-Fi bất đồng bộ, trả danh sách SSID
/saveWifi	Lưu ssid, pass, mqtt_server vào NVS
/reStart	Khởi động lại thiết bị

3.7.4 Nút bấm GPIO15

Nhấn ngắn (dưới 5 giây) bật hoặc tắt chế độ phát cấu hình Wi-Fi qua ESP-NOW, mỗi 500 ms một gói. Giữ từ 5 giây trở lên xóa toàn bộ cấu hình trong NVS rồi khởi động lại.

3.8 Firmware MCU control

3.8.1 Điều khiển bằng cảm ứng

Bảng 3.8.1: Hành vi của hai cảm ứng TTP223B

Cảm ứng	**Thao tác**	**Kết quả**	**Bản tin ESP-NOW gửi đi**
Đèn	Chạm (nhả sau ít nhất 50 ms) khi đèn tắt	Bật bằng màu đã lưu (nếu màu tối thì dùng trắng)	“to_on”
Đèn	Chạm khi đèn bật	Tắt	“to_off”
Quạt	Chạm ngắn khi quạt tắt	Bật ở mức hiện tại (mặc định 50%)	“fan_on_”
Quạt	Chạm ngắn khi quạt chạy	Tắt	“fan_off”
Quạt	Giữ từ 1 giây trở lên	Tăng 25% mỗi giây: 25 → 50 → 75 → 100 → 25	“fan_speed_”

3.8.2 Bộ phân tích lệnh

Lệnh nhận qua ESP-NOW được đưa vào hàng đợi 20 phần tử rồi xử lý theo thứ tự: “OFFRGB”; “ONRGB”; “fan_off”; “fan_on”; chuỗi “R G B”; JSON {r, g, b} (kèm phản hồi “Đã đổi màu: R… G… B…”); “fan_speed_N” hoặc “fan_on_N” với N thuộc {25, 50, 75, 100}. Màu có cả ba thành phần dưới 40 được coi là tắt đèn nhưng không ghi đè màu đã lưu. Màu mới được ghi vào NVS sau khi ổn định 3 giây, để không làm mòn flash khi người dùng kéo thanh trượt liên tục.

3.8.3 Điều khiển quạt bằng PWM

PWM 1 kHz, độ phân giải 8 bit. Mức phần trăm được đổi sang giá trị 0–255 bằng hàm `map`; ở mức 100% chân MOSFET được giữ mức cao liên tục để MOSFET mở hoàn toàn.

Bảng 3.8.3: Mức quạt và giá trị PWM

Mức quạt	**Giá trị PWM (0–255)**	**Độ rộng xung thực tế**
25%	63	≈ 24,7%
50%	127	≈ 49,8%
75%	191	≈ 74,9%
100%	255, chân giữ mức cao	≈ 100%

3.8.4 Tự dò kênh

Khi khởi động, nếu NVS đã có kênh thì MCU dùng kênh đó. Nếu chưa có, hoặc khi giữ nút BOOT từ 5 giây, MCU quét kênh 1–13: trên mỗi kênh gửi "PING_SCAN" rồi chờ 200 ms; MCU center nhận được sẽ phản hồi ACK nên MCU control biết kênh đúng và lưu lại. Một lượt quét tốn tối đa 13×200 ms = 2,6 giây và làm dừng vòng lặp chính.

3.9 Giao thức và định dạng bản tin

Bảng 3.9.1: Gói tin ESP-NOW

Loại	**Giá trị**	**Cấu trúc**	**Kích thước**	**Hướng**	**Nội dung**
Văn bản	0x00	type (1 byte) + text (240 byte)	241 byte	Quảng bá, mọi MCU	Lệnh, dữ liệu DHT11, phản hồi
Cấu hình Wi-Fi	0xA1	type (1) + ssid (33) + password (65) + mqtt_server (40)	139 byte	MCU center tới MCU alexa	Thông tin Wi-Fi và MQTT

Hai bên phải khai báo cấu trúc giống hệt nhau (main.c dùng thuộc tính `packed`, các tệp Arduino dùng mảng ký tự nên không có byte đệm) [9],[10].

Bảng 3.9.2: Topic MQTT

Topic	**Bên phát**	**Bên nhận**	**Nội dung**
esp32/speech	MCU alexa, web app	MCU center	Lệnh giọng nói và lệnh từ web
esp32/rgb	MCU center	Web app	"ONRGB", "OFFRGB", JSON {r, g, b}
esp32/fan	MCU center	Web app	"fan_off", "fan_on_N"
esp32/dht	MCU center	Web app	Nhiệt độ và độ ẩm
esp32/send	MCU center	Web app	Phản hồi, kết quả đọc DHT11 theo yêu cầu
esp32/gemini	Máy chủ AI	MCU recording	Câu trả lời dạng văn bản hoặc JSON {answer}
esp32/curtain, esp32/web	Chưa xác định	Web app	Có trong danh sách đăng ký của web app; chưa có bên phát trong mã đã cung cấp

Bảng 3.9.3: Dữ liệu trên đường UART giữa MCU alexa và MCU recording

Hướng	**Nội dung**	**Mục đích**
alexa → recording	"alexa", "haVcKWfSpcN", "fNDKWfSpcN"	Điều phối đọc phản hồi và ghi âm (xem Bảng 3.6.5)
alexa → recording	Văn bản nhận qua ESP-NOW (to_on, fan_on_N, màu, . . .)	Để MCU recording đọc xác nhận bằng tiếng Việt
alexa → recording	JSON {ssid, pass, mqtt, channel}	Chuyển cấu hình Wi-Fi
recording → alexa	"ready", văn bản câu trả lời, JSON {channel}	Báo trạng thái; MCU alexa hiện chỉ ghi nhật ký

3.10 Đồng bộ kênh và cấu hình Wi-Fi

ESP-NOW và Wi-Fi dùng chung một bộ thu phát nên mọi MCU phải ở cùng kênh. Đề tài giải quyết bằng hai cơ chế:

1. Phát cấu hình từ MCU center. Nhấn nút GPIO15 trên MCU center để phát gói 0xA1 mỗi 500 ms trên kênh của router.
2. Quét kênh ở MCU alexa. Nhấn nút GPIO0 trên MCU alexa: Wi-Fi bị ngắt, chip nhảy kênh 1–13, mỗi kênh dừng 300 ms, LED vàng nhấp nháy, tự hủy sau 30 giây nếu không thấy gói nào.
3. Khi nhận được gói 0xA1, MCU alexa ghi ssid, mật khẩu, địa chỉ MQTT và kênh vào NVS, gửi cấu hình sang MCU recording qua UART (MCU này lưu và khởi động lại), rồi kết nối Wi-Fi và MQTT.
4. MCU control tự dò kênh bằng "PING_SCAN".

Một lượt quét của MCU alexa mất:

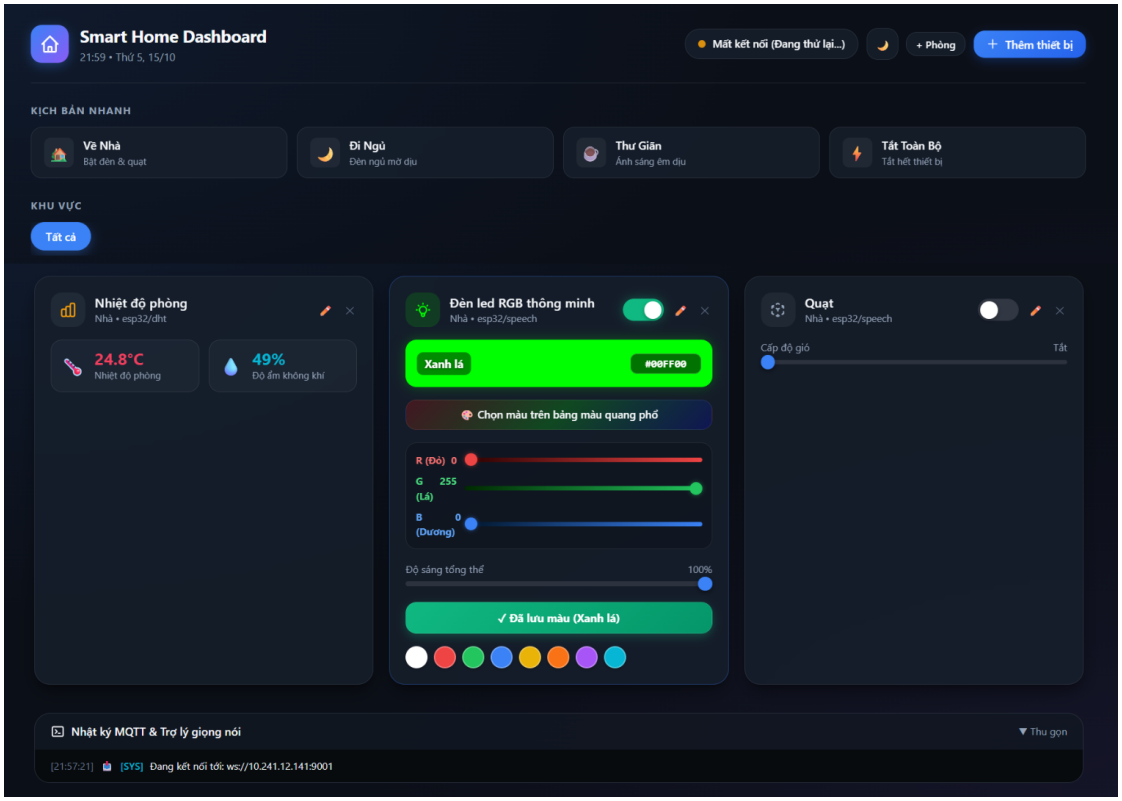
$$13 \times 300\,\text{ms} = 3{,}9\,\text{s}$$

3.11 Giao diện web

Giao diện "Smart Home Dashboard" chạy trên trình duyệt, kết nối MQTT qua WebSocket và gồm các thành phần sau (Hình 3.11):

- **Thanh trạng thái:** Tình trạng kết nối MQTT, nút thêm thiết bị, lọc theo khu vực.
- **Thêm thiết bị:** Giao diện modal cho phép người dùng khai báo thiết bị mới, lựa chọn loại điều khiển (Đèn LED RGB, Quạt, Cảm biến), gán khu vực/phòng và cấu hình các topic MQTT tương ứng.
- **Kịch bản nhanh:** "Về nhà" (bật đèn và quạt), "Đi ngủ" (đèn ngủ mờ dịu), "Thư giãn" (ánh sáng êm dịu), "Tắt toàn bộ".

- **Thẻ nhiệt độ phòng:** Nhiệt độ và độ ẩm lấy từ topic `esp32/dht`.
- **Thẻ đèn LED RGB:** Công tắc, bảng màu quang phổ, ba thanh trượt R, G, B, độ sáng tổng thể, tám màu nhanh và nút "Lưu màu để phát âm" để gắn tên màu cho câu đọc của loa.
- **Thẻ quạt:** Công tắc và thanh chọn cấp độ gió.
- **Nhật ký MQTT và trợ lý giọng nói:** Liệt kê sự kiện đăng ký topic và thông báo hệ thống [14].



Hình 3.11: Giao diện web điều khiển và giám sát

3.11.1 Hướng dẫn các chức năng giao diện Web

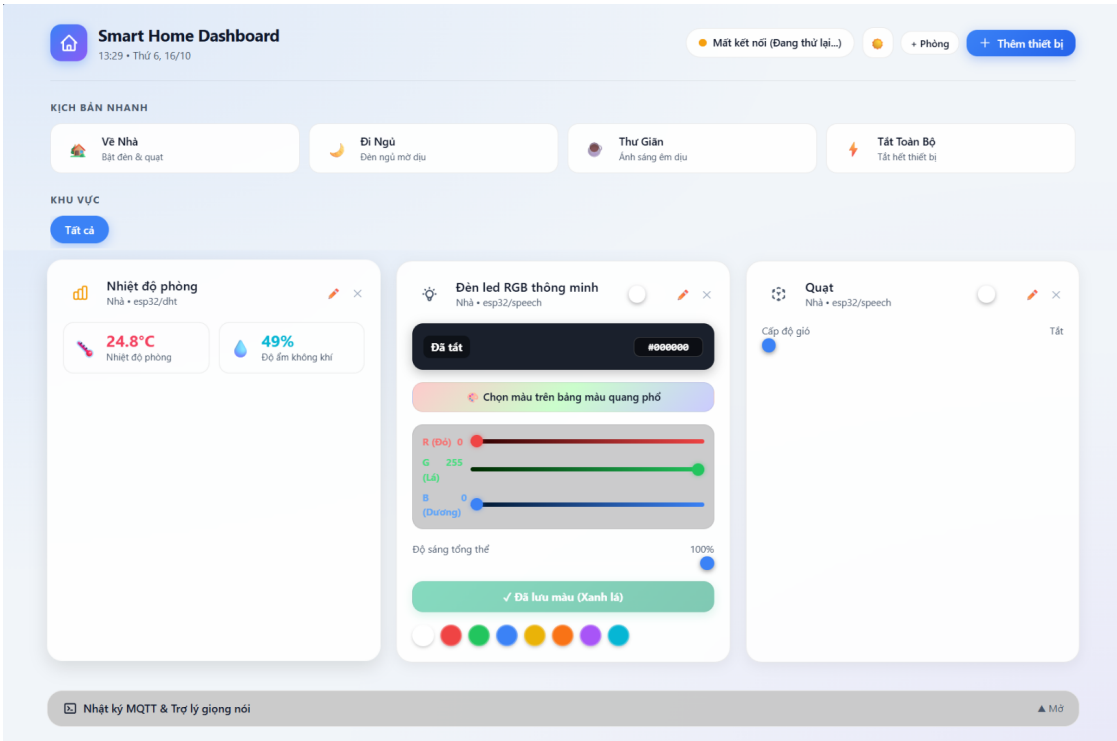
Giao diện giám sát và điều khiển Smart Home Dashboard được thiết kế theo phong cách hiện đại (Modern UI Theme) trên nền tối (Dark mode), tối ưu cho cả trình duyệt máy tính và thiết bị di động. Hệ thống phân chia rõ ràng các khối chức năng chính giúp người dùng thao tác nhanh chóng và chính xác:

1. **Thanh trạng thái và điều hướng hệ thống (Header Bar):**
   - **Cấu hình kết nối MQTT Broker:** Nhấn vào thẻ trạng thái (*Status Badge*) ở góc trên để mở hộp thoại cấu hình địa chỉ WebSocket Broker (`ws://10.251.10.110:9001` hoặc `ws://localhost:9001`). Thẻ sẽ tự động thay đổi màu sắc để biểu thị trạng thái (Xanh lá: Đã kết nối, Vàng: Đang kết nối, Đỏ: Mất kết nối).



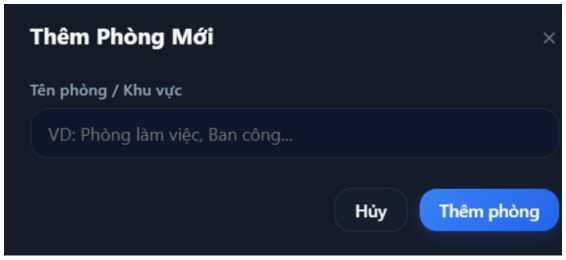
Hình 3.11.1.1: Giao diện đổi MQTT

- **Chuyển đổi giao diện (Theme):** Nút biểu tượng Mặt trăng/Mặt trời cho phép chuyển đổi nhanh giữa chế độ Sáng Hình 3.11.1.2 (Light Mode) và Tối Hình 3.11 (Dark Mode).

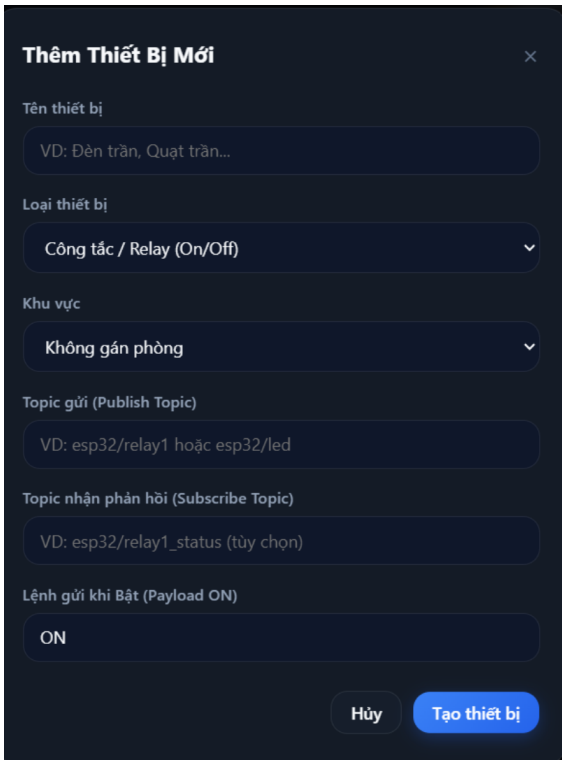


Hình 3.11.1.2: Giao diện web điều khiển và giám sát (Light Mode).

- **Quản lý khu vực và thiết bị:** Cung cấp hai nút bấm nhanh `+ Phòng` để khởi tạo khu vực mới và `+ Thêm thiết bị` để khai báo thiết bị phần cứng vào hệ thống.



Hình 3.11.1.3: Giao diện thêm phòng mới



Hình 3.11.1.4: Giao diện thêm thiết bị mới

2. **Kích hoạt kịch bản nhanh (Quick Scenes):** Cho phép điều khiển đồng bộ nhiều thiết bị cùng lúc chỉ với một thao tác chạm.

3. **Lọc thiết bị theo khu vực (Rooms Bar):** Thanh Tab phân loại hỗ trợ lọc danh sách hiển thị theo từng phòng cụ thể (Phòng khách, Phòng ngủ, Sân vườn...) hoặc chọn tab `Tất cả` để quản lý tập trung toàn bộ thiết bị.

4. **Khối giám sát và điều khiển thiết bị (Devices Grid):**

   - **Thẻ Nhiệt độ phòng (Sensor Card):** Cập nhật dữ liệu thời gian thực từ cảm biến DHT11 thông qua topic `esp32/dht`, hiển thị trực quan thông số Nhiệt độ (°C) và Độ ẩm không khí (%).
   - **Thẻ Đèn LED RGB Thông Minh:**
     - **Công tắc nguồn:** Bật/tắt nhanh trạng thái đèn.
     - **Thanh xem trước màu sắc (Banner):** Hiển thị tên màu nhận diện và mã Hex (cho phép nhập trực tiếp mã màu dạng `#00FF00`).

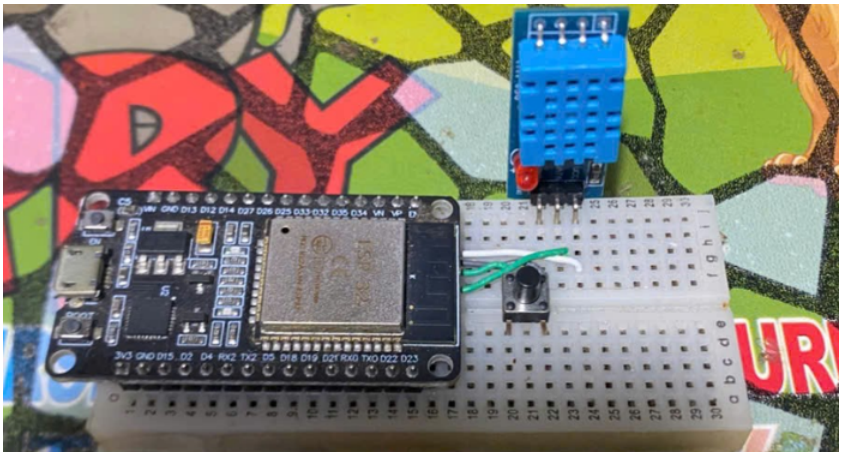
  - **Bảng màu quang phổ (Color Picker):** Cho phép chọn bất kỳ màu sắc mong muốn từ bảng màu đồ họa.
  - **Bộ trộn kênh R-G-B:** Điều chỉnh tinh chỉnh 3 thanh trượt Red (0–255), Green (0–255), Blue (0–255) và thanh điều chỉnh **Độ sáng tổng thể** (1–100%).
  - **Nút "Lưu màu để phát âm":** Khi bấm lưu, hệ thống không chỉ lưu trạng thái màu mà còn tự động trích xuất tên màu tiếng Việt tương ứng (VD: *"Xanh lá"*, *"Đỏ"*, *"Tím"*) và gửi bản tin qua topic `esp32/speech` để phục vụ phản hồi bằng giọng nói.
  - **Dải màu nhanh (Presets):** Cung cấp 8 ô màu cố định (Trắng, Đỏ, Xanh lá, Xanh dương, Vàng, Cam, Tím, Xanh ngọc) để đổi màu tức thì.
- **Thẻ Điều khiển Quạt (Fan Card):** Tích hợp công tắc nguồn và thanh trượt 4 cấp độ gió (25%, 50%, 75%, 100%) tương ứng với các lệnh PWM gửi tới MCU.

5. **Nhật ký MQTT & Trợ lý giọng nói (Activity Console):** Cửa sổ console ở chân trang ghi nhận liên tục toàn bộ lưu lượng bản tin MQTT truyền/nhận trong hệ thống dưới dạng thời gian thực (Timestamp, Topic, Payload). Đồng thời hỗ trợ theo dõi các câu lệnh nhận dạng giọng nói vừa thực thi, giúp quản trị viên dễ dàng kiểm tra và gỡ lỗi (debug).

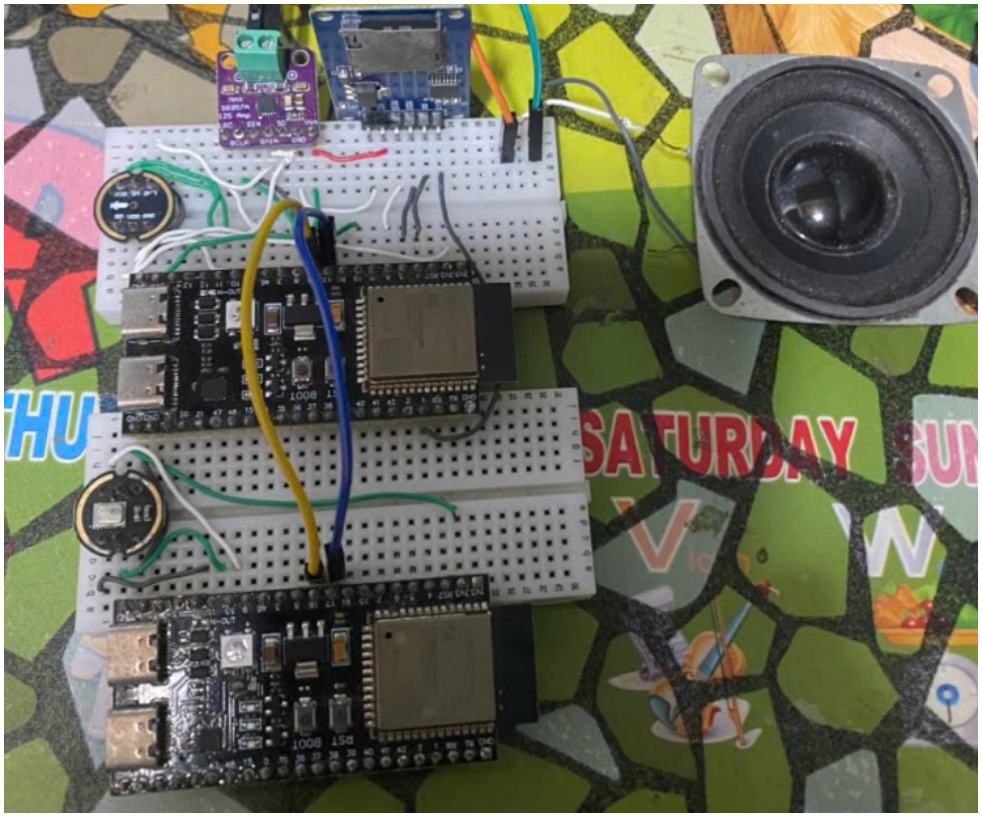
CHƯƠNG 4: KẾT QUẢ THỰC HIỆN VÀ KIỂM THỬ

4.1 Kết quả thực nghiệm và kiểm thử

Mô hình đã được lắp trên breadboard thành ba khối phần cứng và hoạt động đủ bốn chức năng: điều khiển đèn và quạt bằng giọng nói tiếng Anh, hỏi – đáp và phản hồi bằng tiếng Việt, điều khiển tại chỗ bằng cảm ứng và điều khiển qua ứng dụng web. Video demo: `https://youtube.com/shorts/wCcJCsxGOXc`.



Hình 4.1: MCU center (ESP32 DevKit V1) với cảm biến DHT11 và nút nhấn



Hình 4.2: Hai ESP32-S3: MCU alexa và MCU recording kèm micro, thẻ SD, module âm thanh và loa.

Bảng 4.1: Đối chiếu kết quả với mục tiêu

Mục tiêu (mục 1.3)	**Kết quả**	**Ghi chú**
Xử lý giọng nói hai tầng	Lệnh tiếng Anh nhận dạng ngoại tuyến; câu hỏi tiếng Việt gửi lên Gemini	Câu hỏi về tin tức mới chưa có dữ liệu
Phản hồi bằng TTS	Đọc được câu trả lời tiếng Việt, có hàng đợi chia đoạn	
Truyền thông ESP-NOW và MQTT	Hoạt động, có cơ chế đồng bộ kênh và phát cấu hình	Chưa mã hóa ESP-NOW
Mô hình đại diện	Đèn RGB và quạt DC điều khiển được bằng giọng nói, cảm ứng và web	
Ứng dụng trên thiết bị di động (phiếu đăng ký)	Web app chạy trên trình duyệt	Chưa đóng gói thành ứng dụng, chưa có cơ sở dữ liệu

4.2 Phương pháp kiểm thử

Kiểm thử chia hai mức. Verification kiểm tra từng yêu cầu ở Bảng 3.1.1 và 3.1.2 có được thỏa không, dựa trên nhật ký Serial 115 200 baud của từng MCU và nhật ký MQTT trên web app. Validation kiểm tra người dùng có dùng được hệ thống để điều khiển thiết bị như mong đợi hay không. Độ trễ được đo bằng video 60 khung/giây, đếm khung hình từ lúc người nói dứt lệnh đến lúc LED hoặc quạt đổi trạng thái. Mỗi lệnh thử lặp lại nhiều lần và ghi số liệu vào Bảng 4.3 và Bảng 4.4.

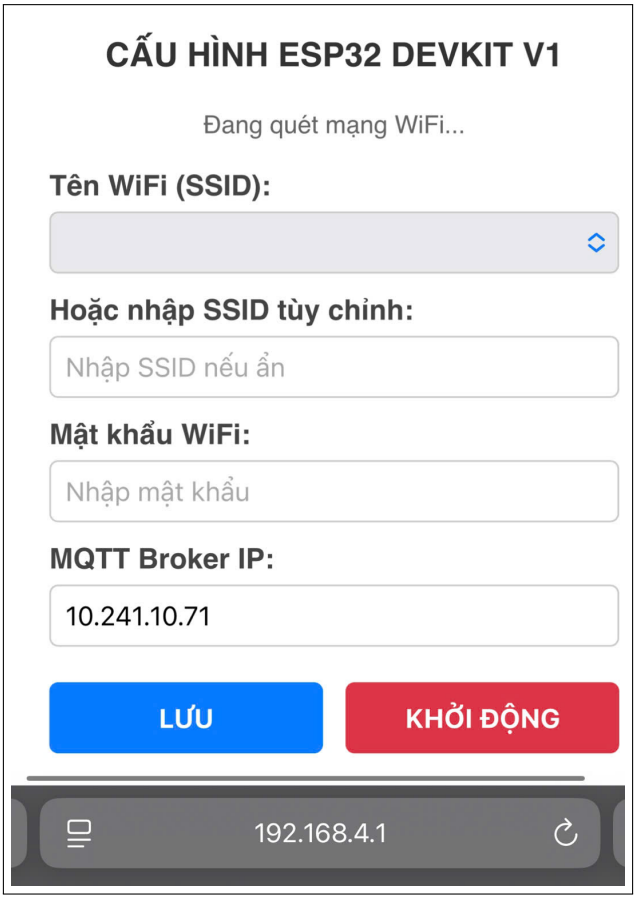
4.3 Kịch bản kiểm thử và kết quả [7]

Bảng 4.2: Kịch bản kiểm thử chức năng

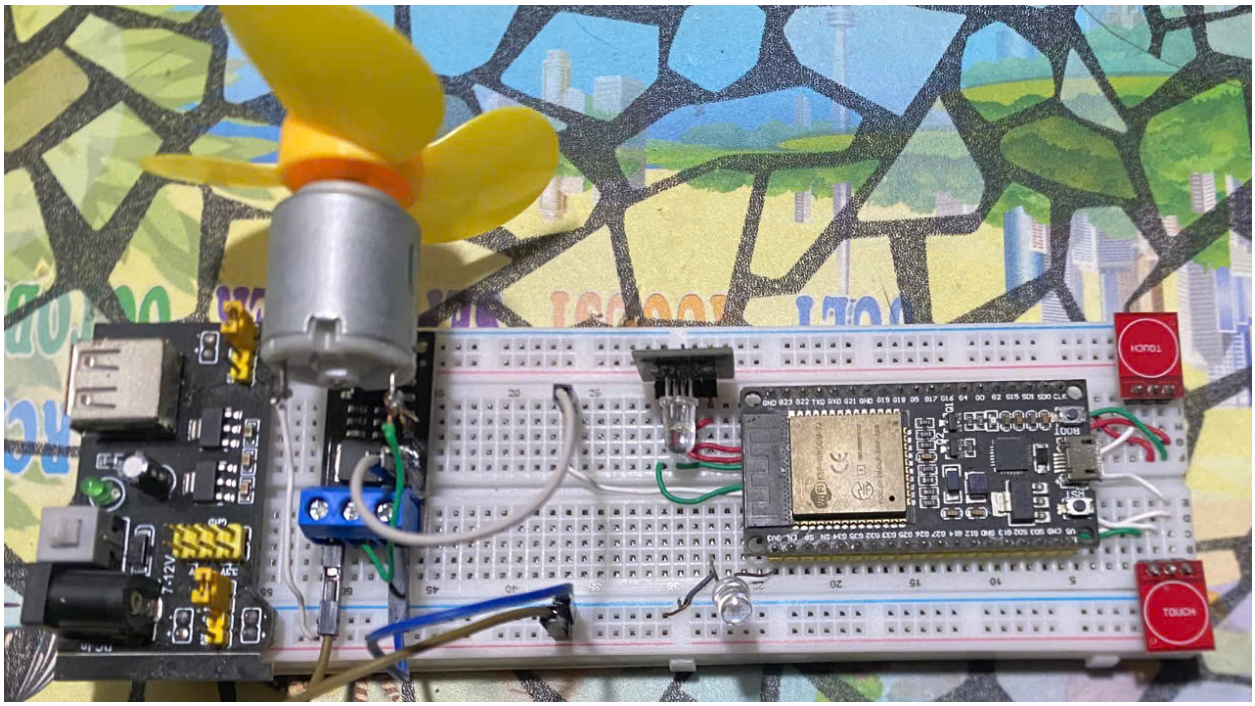
Mã	**Yêu cầu**	**Cách thử**	**Kết quả mong đợi**	**Kết quả thực tế**
TC01	FR1	Nói “alexa” cách micro 0,5 m trong phòng yên tĩnh, lặp 10 lần	LED trắng mờ, loa đọc “Dạ, em nghe”	Đạt
TC02	FR2	Mỗi lệnh trong Bảng 3.1 nói 10 lần	Nhận dạng đúng và gửi đúng bản tin	Đèn và quạt điều khiển được bằng giọng nói; Đạt
TC03	FR2, NFR1	Ngắt đường ra Internet, giữ Wi-Fi nội bộ, nói lệnh bật đèn	Đèn sáng	Đạt

(Tiếp theo Bảng 4.2)

Mã	Yêu cầu	Cách thử	Kết quả mong đợi	Kết quả thực tế
TC04	FR2	Nói lệnh đổi màu theo tên màu (xanh lá, đỏ, xanh dương)	Đèn đổi đúng màu	Màu sắc bị giới hạn với số màu đã cài sẵn; Đạt một phần
TC05	FR3	Hỏi một câu tiếng Việt khoảng 5–10 giây	Loa đọc câu trả lời	Đạt
TC06	FR4	Đổi màu bằng web; cấp lại nguồn MCU control	Đèn đổi màu; sau cấp nguồn dùng lại màu gần nhất	Đổi màu bằng web: đạt; nhớ màu lưu gần nhất hoặc mặc định là trắng
TC07	FR5	Đặt quạt ở 25, 50, 75, 100%	Tốc độ quạt tăng dần	Đạt;
TC08	FR6	Chạm ngắn và giữ từng cảm ứng	Đèn/quạt đổi trạng thái theo Bảng 3.8.1	Đạt
TC9	FR7	Quan sát topic esp32/dht	Mỗi 6 giây một bản tin; web hiển thị	Đạt: web hiển thị 30,8 °C và 72,9%
TC10	FR8	Mở web app	Thấy trạng thái MQTT, thiết bị, nhật ký	Đạt
TC11	FR9	Cấu hình qua AP; phát cấu hình bằng ESP-NOW	MCU alexa nhận được Wi-Fi và MQTT	Web config: đạt; phát bằng ESP-NOW: Đạt
TC12	FR10, NFR3	Tắt router, dùng cảm ứng và giọng nói	Lệnh vẫn tới MCU control nếu cùng kênh	Đạt



Hình 4.3: Trang web cấu hình Wi-Fi và MQTT (192.168.4.1).



Hình 4.4: MCU control với hai cảm ứng, LEDRGB, MOSFET và quạt.

Bảng 4.3: Độ trễ từ lúc dứt lệnh đến lúc thiết bị đổi trạng thái

Con đường	**Số lần đo**	**Trung bình**
Giọng nói → ESP-NOW → quạt	10	≤ 1 s
Giọng nói → MCU center → đèn bật/tắt	10	≤ 1 s
Web app → MQTT → MCU center → ESP-NOW → đèn	10	≤ 1 s
Cảm ứng → ESP-NOW → phản hồi TTS	10	≤ 1 s

Bảng 4.4: Tỷ lệ nhận dạng lệnh theo khoảng cách

Điều kiện	**Số lần nói**	**Số lần đúng**	**Tỷ lệ**
0,5 m, phòng yên tĩnh	10	9	90%
1 m, phòng yên tĩnh	10	7	70%
2 m, phòng yên tĩnh	10	5	50%
1 m, có tiếng ồn nền (quạt chạy)	10	4	40%

4.4 Các vấn đề gặp phải và cách xử lý

Trong quá trình phát triển, nhiều lỗi chỉ lộ ra khi chạy trên phần cứng thật. Bảng 4.5 liệt kê các lỗi đã được sửa trong mã nguồn.

Bảng 4.5: Vấn đề gặp phải, nguyên nhân và giải pháp

Vấn đề	**Nguyên nhân**	**Giải pháp**
Không đánh thức được bằng “alexa” sau khi quét kênh	Quét kênh không có giới hạn thời gian, cờ `esp_now_rx_mode` kẹt ở true và `detect_Task` bỏ qua nhận dạng	Thêm thời gian tự hủy 30 giây; sau quét luôn đặt lại chế độ chờ “alexa”
Wi-Fi rớt khi đổi kênh	Gọi `esp_wifi_set_channel()` khi đang nối router, kênh phải theo router	Chỉ ép kênh khi chưa nối router; khi đã nối dùng kênh hiện tại (`peer.channel = 0`)
TTS im lặng với câu trả lời dài	Dịch vụ TTS giới hạn khoảng 200 ký tự mỗi yêu cầu	Chia đoạn ≤ 150 byte, gộp câu ngắn, xếp hàng đợi (mục 3.6.4)
Màu đèn bị ghi đè khi nói “bật đèn” kèm tên màu	Hai bản tin đến gần nhau, bản chung ghi đè bản cụ thể	Bỏ qua lệnh chung trong 1,5 giây sau lệnh màu; chờ 400 ms trước khi bật lại màu cũ
Đứng màn hình khi ghi nhiều lần vào flash	Mỗi lần kéo thanh trượt màu đều ghi NVS	Chỉ ghi sau khi màu ổn định 3 giây
Tràn hàng đợi ESP-NOW	Nhiều bản tin đến cùng lúc	Hàng đợi 20 phần tử, đếm số gói bị bỏ (`rxQueueDropCount`)
MCU bị treo khi nhận cấu hình sai	Chuỗi rỗng hoặc thiếu trường JSON	Kiểm tra con trỏ rỗng trước khi lưu cấu hình (MCU alexa)

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được

Đề tài đã thiết kế và thi công được một bộ điều khiển trung tâm gồm bốn vi điều khiển cộng tác với nhau, chạy thật trên phần cứng và đã được trình diễn bằng video. Các kết quả chính:

1. **Phần cứng:** Hoàn thành mô hình gồm MCU alexa, MCU recording, MCU center và MCU control; micro INMP441, loa MAX98357A, cảm biến DHT11, nút chạm TTP223B, quạt (MOSFET AOD4184) và đèn RGB đều hoạt động theo sơ đồ nối chân ở mục 3.3.
2. **Nhận dạng giọng nói hai tầng:** Lệnh điều khiển cơ bản (lệnh tiếng Anh) được nhận dạng ngay trên chip bằng ESP-SR, không cần Internet; câu hỏi tự do được ghi âm, gửi lên máy chủ và trả lời bằng Gemini rồi đọc lại bằng giọng tiếng Việt.
3. **Giao tiếp hai kênh:** ESP-NOW cho đường điều khiển ưu tiên độ trễ thấp; MQTT cho đồng bộ với giao diện web và dự phòng. MCU center làm cầu nối giữa hai kênh.
4. **Tự cấu hình:** Cấu hình Wi-Fi qua trang web của MCU center (AP TDTHINH-xxxx, địa chỉ `192.168.4.1`), tự phát lại cho các MCU còn lại bằng gói 0xA1; quét kênh vô tuyến tự động; lưu màu và trạng thái vào NVS.
5. **Giao diện web:** Xem nhiệt độ, độ ẩm và điều khiển quạt, đèn từ trình duyệt trên điện thoại.

5.2 Hạn chế

Bảng 5.1: Hạn chế hiện tại và hướng khắc phục

Hạn chế	**Nguyên nhân / ảnh hưởng**	**Hướng khắc phục**
ESP-NOW broadcast [9, 10], không ACK, không mã hóa	Gói có thể mất; thiết bị lạ trong tầm sóng có thể gửi lệnh giả	Gửi lặp, thêm ACK và số thứ tự; bật mã hóa PMK/LMK
MQTT chưa xác thực	Ai biết địa chỉ broker đều đọc và gửi được bản tin	Dùng tài khoản riêng, TLS, giới hạn quyền theo topic
Mật khẩu Wi-Fi ở dạng rõ (trang cấu hình, UART, nhật ký)	Có thể lộ khi nghe trộm đường truyền hoặc đọc nhật ký	Mã hóa khi truyền, không in mật khẩu ra nhật ký
Lệnh đổi màu bằng giọng nói ở MCU center mới là khung	Đổi màu hiện chủ yếu qua nút chạm và web	Hoàn thiện hàm xử lý lệnh màu và kiểm thử

Xem tiếp trang sau...

Bảng 5.1: Hạn chế hiện tại và hướng khắc phục (*tiếp theo*)

Hạn chế	Nguyên nhân / ảnh hưởng	Hướng khắc phục
Không trả lời được thông tin mới (tin tức, thời tiết)	Gemini chỉ dựa vào dữ liệu huấn luyện	Bật Google Search grounding ở máy chủ
Lệnh ngoại tuyến chỉ có tiếng Anh	ESP-SR chưa có mô hình tiếng Việt dựng sẵn	Huấn luyện/tùy biến mô hình lệnh tiếng Việt
Giao diện web chưa đóng gói thành ứng dụng, chưa có cơ sở dữ liệu	Không lưu lịch sử; phải mở trình duyệt	Đóng gói PWA/ứng dụng di động, thêm cơ sở dữ liệu chuỗi thời gian
Thời gian khởi động và quét kênh chưa tối ưu	Quét 13 kênh, tối đa 30 s trước khi vào hoạt động ổn định	Lưu kênh gần nhất vào NVS và thử kênh đó trước

Các hạn chế trên là cố ý chấp nhận ở giai đoạn nguyên mẫu để ưu tiên chạy được luồng đầy đủ; chúng được đưa vào hướng phát triển ở mục 5.3.

5.3 Hướng phát triển

1. **Bảo mật:** Mã hóa gói ESP-NOW, bật xác thực và TLS cho MQTT, không để mật khẩu Wi-Fi ở dạng rõ trong UART và nhật ký.
2. **Hoàn thiện điều khiển bằng giọng nói:** Cài đặt xử lý lệnh đổi màu ở MCU center; mở rộng bộ lệnh; thử nhận dạng lệnh tiếng Việt ngoại tuyến bằng mô hình tùy chỉnh.
3. **Trả lời thông tin mới:** Bật công cụ tìm kiếm của Gemini (Google Search grounding) để trả lời tin tức, thời tiết, giá cả.
4. **Ứng dụng di động và cơ sở dữ liệu:** Đóng gói giao diện web thành ứng dụng trên điện thoại; lưu lịch sử nhiệt độ, độ ẩm và nhật ký lệnh để vẽ biểu đồ.
5. **Mở rộng thiết bị:** Thêm các nút điều khiển rèm cửa, ổ cắm, khóa cửa; cho phép thêm nút mới mà không sửa mã nhờ bản tin tự giới thiệu.
6. **Phần cứng:** Thiết kế mạch in (PCB) tích hợp, tối ưu thời gian khởi động và tiêu thụ điện (chế độ ngủ cho các nút ít dùng).

5.4 Kết luận

Đề tài cho thấy khả năng xây dựng bộ điều khiển trung tâm nhà thông minh với chi phí tương đối thấp trên nền tảng vi điều khiển ESP32, kết hợp công nghệ nhận dạng giọng nói ngoại tuyến với các dịch vụ trí tuệ nhân tạo trên nền tảng đám mây. Mô hình đã triển khai các chức năng chính như điều khiển thiết bị bằng giọng nói, hỗ trợ hỏi đáp bằng trí tuệ nhân tạo, trao đổi dữ liệu giữa các vi điều khiển thông qua ESP-NOW, MQTT và UART, đồng thời giám sát và điều khiển thiết bị thông qua giao diện web. Tuy nhiên, để có thể

ứng dụng rộng rãi trong thực tế, hệ thống vẫn cần tiếp tục được hoàn thiện về hiệu năng, độ tin cậy, khả năng bảo mật và kiểm thử định lượng.

Bên cạnh kết quả nghiên cứu và triển khai mô hình, đề tài còn cung cấp một nền tảng tham khảo cho sinh viên trong quá trình học tập, nghiên cứu và phát triển các hệ thống nhà thông minh. Từ kiến trúc phần cứng, phương thức giao tiếp giữa các vi điều khiển đến việc tích hợp nhận dạng giọng nói, trí tuệ nhân tạo và giao diện giám sát, sinh viên có thể kế thừa, mở rộng và phát triển mô hình thành một hệ thống Smart Home hoàn chỉnh hơn. Các hướng phát triển có thể bao gồm bổ sung nhiều loại cảm biến và thiết bị điện, xây dựng các kịch bản tự động hóa theo điều kiện môi trường, lịch trình hoặc thói quen sử dụng, quản lý thiết bị tập trung, giám sát dữ liệu theo thời gian thực, điều khiển từ xa qua Internet và tăng cường bảo mật cho hệ thống.

Trong tương lai, mô hình có thể tiếp tục được phát triển theo hướng tích hợp đầy đủ các chức năng của một hệ thống nhà thông minh thực tế, bảo đảm khả năng mở rộng và thuận tiện cho người sử dụng. Qua đó, đề tài không chỉ là một mô hình nghiên cứu và thực nghiệm mà còn có thể trở thành cơ sở để sinh viên tiếp tục nghiên cứu, sáng tạo và xây dựng các giải pháp Smart Home phù hợp với nhu cầu sử dụng thực tế.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems. ESP32 series datasheet. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[2] Analog Devices. MAX98357A/MAX98357B – tiny, low-cost, PCM class D amplifier with class AB performance. [Online]. Available: https://www.analog.com/en/products/max98357a.html

[3] Aosong (Guangzhou) Electronics. DHT11 humidity & temperature sensor datasheet. [Online]. Available: https://datasheet.lcsc.com/szlcsc/Aosong-Guangzhou-Elec-DHT11_C117051.pdf

[4] Components101. TTP223 single channel touch detector IC. [Online]. Available: https://components101.com/ics/ttp223-single-channel-touch-detector-ic

[5] Alpha & Omega Semiconductor. AOD4184A – 40v n-channel MOSFET datasheet. [Online]. Available: https://www.aosmd.com/res/datasheets/AOD4184A.pdf

[6] schreibfaul1. ESP32-audioI2S. [Online]. Available: https://github.com/schreibfaul1/ESP32-audioI2S

[7] F. P. Wedyanti and A. Wagyana, "Analisis performasi ESP-NOW sebagai protokol komunikasi efisien antar multi-node dan gateway dalam sistem IoT," in *Seminar Nasional Inovasi Vokasi (SNIV)*, vol. 4, no. 1, 2025, pp. 607–614. [Online]. Available: https://prosiding.pnj.ac.id/index.php/sniv/article/download/3937/2329

[8] T. Project. (2023) We can create a voice command system! Đăng ngày 26 thg 5, 2023. [Online]. Available: https://youtu.be/3XbnzfBjmZk?si=JMrS7dj9uxPYCpZK

[9] Espressif Systems. ESP-NOW ESP-IDF programming guide (api reference). [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/api-reference/network/esp_now.html

[10] E. Systems. ESP-NOW Programming Guide. [Online]. Available: https://docs.espressif.com/projects/esp-now/en/latest/esp32/index.html

[11] OASIS. MQTT version 3.1.1 – OASIS standard. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[12] Espressif Systems. ESP-SR user guide. [Online]. Available: https://docs.espressif.com/projects/esp-sr/en/latest/esp32/esp-sr-en-master-esp32.pdf

[13] Google. Gemini API – text generation. [Online]. Available: https://ai.google.dev/gemini-api/docs/text-generation

[14] MQTT.js contributors. MQTT.js. [Online]. Available: https://github.com/mqttjs/MQTT.js

A MÃ NGUỒN

Phụ lục này trích các đoạn mã tiêu biểu từ bốn firmware và giao diện web. Mã được lấy nguyên văn từ tệp nguồn; chỉ bỏ dấu tiếng Việt và ký tự đặc biệt trong chú thích và chuỗi in ra để biên dịch được. Toàn bộ mã nguồn xem tại kho mã của đề tài: https://github.com/tranthinhcv09/NCKH_trungtamdieukhienthietbi.

A.1 Danh mục tệp mã nguồn

Bảng A.1: Danh mục tệp mã nguồn của đề tài

Tệp	Thuộc khối / vai trò	Môi trường	Số dòng
`main.c`	MCU alexa: nhận dạng từ khóa và lệnh (ESP-SR), phát lệnh qua ESP-NOW/MQTT, điều phối UART	ESP-IDF v4.4.8	1 035
`NCKHa8.ino`	MCU recording: ghi âm, tải lên máy chủ AI, đọc câu trả lời bằng TTS	Arduino-ESP32	923
`NCKHb2.ino`	MCU center: cầu nối ESP-NOW ↔ MQTT, đọc DHT11, trang web cấu hình	Arduino-ESP32	831
`NCKHc6.ino`	MCU control: cảm ứng, PWM quạt, đèn RGB, lưu NVS, tự dò kênh	Arduino-ESP32	604
`index.html`	Giao diện web: bố cục, hộp thoại	HTML	435
`rgb.js`	Giao diện web: kết nối MQTT, điều khiển thiết bị, lưu trạng thái	JavaScript (MQTT.js)	1 705
`style.css`	Giao diện web: giao diện sáng/tối, bố cục đáp ứng	CSS	1 166

A.2 Khai báo gói tin ESP-NOW (MCU alexa, ESP-IDF)

Hai cấu trúc gói tin dùng chung giữa các vi điều khiển (Bảng 3.9.1). Thuộc tính `packed` giúp kích thước gói đúng 241 byte và 139 byte.

```
#define ESPNOW_MSG_WIFI_CONFIG  0xA1
#define ESPNOW_MSG_TEXT         0x00

typedef struct __attribute__((packed)) {
    uint8_t type;           // 0xA1
    char ssid[33];
    char password[65];
    char mqtt_server[40];
} wifi_credentials_message_t;

typedef struct __attribute__((packed)) {
    uint8_t type;           // 0x00
    char text[240];
} esp_now_text_message_t;
```

Đoạn mã A.1: Cấu trúc gói tin ESP-NOW trên MCU alexa (main.c)

A.3 Gửi lệnh ưu tiên ESP-NOW, dự phòng MQTT

Hàm `esp_now_publish_text` gửi quảng bá và chờ trạng thái gửi tối đa 200 ms; hàm `publish_text_priority` chuyển sang MQTT (QoS 1) nếu ESP-NOW thất bại (mục 3.5.3).

```
static bool esp_now_publish_text(const char *text) {
    if (!text || !esp_now_active || esp_now_rx_mode || !s_esp_now_send_sem) return false;

    if (esp_now_is_peer_exist(s_broadcast_mac)) {
        esp_now_peer_info_t peer = {0};
        memcpy(peer.peer_addr, s_broadcast_mac, 6);
        peer.channel = 0; // dung kenh hien tai cua radio
        peer.encrypt = false;
        peer.ifidx = ESP_IF_WIFI_STA;
        esp_now_mod_peer(&peer);
    }

    esp_now_text_message_t msg = {0};
    msg.type = ESPNOW_MSG_TEXT;
    strncpy(msg.text, text, sizeof(msg.text) - 1);

    s_esp_now_last_send_ok = false;
    xSemaphoreTake(s_esp_now_send_sem, 0);

    esp_err_t err = esp_now_send(s_broadcast_mac, (const uint8_t *)&msg, sizeof(msg));
    if (err != ESP_OK) {
        printf(" ESP-NOW send error: %d (%s)\n", err, esp_err_to_name(err));
        return false;
    }

    if (xSemaphoreTake(s_esp_now_send_sem, pdMS_TO_TICKS(200)) != pdTRUE) {
        printf(" ESP-NOW timeout: khong nhan duoc ACK gui di trong 200ms\n");
        return false;
    }

    return s_esp_now_last_send_ok;
}
```

Đoạn mã A.2: Gửi quảng bá ESP-NOW (main.c)

```
static void publish_text_priority(const char *topic, const char *text) {
    if (!text) return;

    bool sent_via_espnow = esp_now_publish_text(text);

    if (sent_via_espnow) {
        printf(" [TX ESP-NOW KENH %d SUCCESS]: %s\n", g_esp_now_channel, text);
    } else {
        if (mqtt_connected) {
            esp_mqtt_client_publish(mqtt_client, topic, text, 0, 1, 0);
            printf(" [FALLBACK MQTT SUCCESS]: %s\n", text);
```

```
        } else {
            printf(" [TX FAILED] ESP-NOW/MQTT khong kha dung: %s\n", text);
        }
    }
}
```

Đoạn mã A.3: Hàm gửi ưu tiên ESP-NOW và dự phòng MQTT (main.c)

A.4 Tác vụ nhận dạng giọng nói

Phần đầu của `detect_Task`: khởi tạo MultiNet với thời gian chờ 6 000 ms, nạp danh sách lệnh từ `sdkconfig`, và xử lý khi WakeNet phát hiện từ khóa (mục 3.5.1).

```
void detect_Task(void *arg) {
    esp_afe_sr_data_t *afe_data = arg;
    int afe_chunksize = afe_handle->get_fetch_chunksize(afe_data);
    char *mn_name = esp_srmodel_filter(models, ESP_MN_PREFIX, ESP_MN_ENGLISH);
    esp_mn_iface_t *multinet = esp_mn_handle_from_name(mn_name);
    model_iface_data_t *model_data = multinet->create(mn_name, 6000);
    int mu_chunksize = multinet->get_samp_chunksize(model_data);
    esp_mn_commands_update_from_sdkconfig(multinet, model_data);
    assert(mu_chunksize == afe_chunksize);

    bool is_question_mode = false;
    TickType_t question_start_tick = 0;

    printf("------------ Say alexa to start ------------\n");
    stop_channel_scan();
    esp_now_rx_mode = false;
    set_esp_now_active(false);

    while (task_flag) {
        afe_fetch_result_t *res = afe_handle->fetch(afe_data);
        if (!res || res->ret_value == ESP_FAIL) break;

        if (esp_now_rx_mode) {
            vTaskDelay(pdMS_TO_TICKS(10));
            continue;
        }

        if (res->wakeup_state == WAKENET_DETECTED) {
            set_esp_now_active(true);
            led_set_color(80, 80, 80);
            mqtt_publish_speech("alexa");
            uart_send_text("alexa \n");
            printf(" WAKEWORD alexa detected!\n");
            multinet->clean(model_data);
            detect_flag = 1;
            is_question_mode = false;
        }
```

```
38
39          if (detect_flag == 1) {
40              esp_mn_state_t mn_state = multinet->detect(model_data, res->data);
41
42              if (mn_state == ESP_MN_STATE_DETECTING) {
43                  continue;
```

Đoạn mã A.4: Tác vụ detect_Task (trích đoạn, main.c)

A.5 Điều khiển tốc độ quạt bằng PWM

Mức 100% giữ chân MOSFET ở mức cao liên tục; các mức khác đổi sang 0–255 bằng hàm `map` (Bảng 3.8.3).

```
1  void applyFanSpeed(uint8_t percent) {
2      if (!fanOn || percent == 0) {
3          ledcWrite(FAN_MOSFET_PIN, 0);
4          digitalWrite(FAN_MOSFET_PIN, LOW); // Dam bao ngat hoan toan MOSFET
5      } else {
6          if (percent == 100) {
7              // Khi 100%, xuat muc HIGH truc tiep giup MOSFET mo toi da
8              digitalWrite(FAN_MOSFET_PIN, HIGH);
9              ledcWrite(FAN_MOSFET_PIN, 255);
10         } else {
11             uint32_t pwmVal = map(percent, 0, 100, 0, 255);
12             ledcWrite(FAN_MOSFET_PIN, pwmVal);
13         }
14     }
15 }
16
17 // ============================================================
```

Đoạn mã A.5: Hàm applyFanSpeed trên MCU control (NCKHc6.ino)

A.6 Tải tệp ghi âm lên máy chủ AI

Gửi HTTP/1.1 POST dạng `multipart/form-data` tới `/voice` (cổng 5000), đọc từng khối 1 024 byte từ thẻ SD, chờ phản hồi tối đa 10 giây (mục 3.6.3).

```
1  // ===== UPLOAD HTTP =====
2  String uploadAudioAndGetText(const char *path) {
3    File f = SD.open(path, FILE_READ);
4    if (!f) return "";
5
6    WiFiClient httpClient;
7    if (!httpClient.connect(serverIp, serverPort)) {
8      f.close();
9      return "";
10   }
11
12   String boundary = "----ESP32S3Boundary";
```

```
  String bodyStart = "--" + boundary +
                     "\r\nContent-Disposition: form-data; name=\"file\"; filename=\"rec.wav\""
                     "\r\nContent-Type: audio/wav\r\n\r\n";
  String bodyEnd = "\r\n--" + boundary + "--\r\n";
  size_t totalLen = bodyStart.length() + f.size() + bodyEnd.length();

  httpClient.printf(
      "POST %s HTTP/1.1\r\nHost: %s\r\n"
      "Content-Type: multipart/form-data; boundary=%s\r\n"
      "Content-Length: %u\r\nConnection: close\r\n\r\n",
      uploadPath, serverIp, boundary.c_str(), (unsigned)totalLen
  );

  httpClient.print(bodyStart);
  uint8_t buffer[1024];
  while (f.available()) {
    int n = f.read(buffer, sizeof(buffer));
    if (n <= 0) break;
    httpClient.write(buffer, n);
  }
  httpClient.print(bodyEnd);
  f.close();

  unsigned long timeout = millis();
  while (httpClient.connected() && !httpClient.available()) {
    if (millis() - timeout > 10000) { httpClient.stop(); return ""; }
    delay(10);
  }

  while (httpClient.connected()) {
    String line = httpClient.readStringUntil('\n');
    if (line == "\r") break;
  }

  String responseBody = httpClient.readString();
  httpClient.stop();

  JsonDocument doc;
  DeserializationError error = deserializeJson(doc, responseBody);
  if (!error) {
    const char* answer = doc["answer"];
    if (answer) return String(answer);
  }
  return "";
```

Đoạn mã A.6: Hàm uploadAudioAndGetText trên MCU recording (NCKHa8.ino)

A.7 Hàng đợi đọc câu trả lời

Tách câu theo dấu kết thúc câu (không tách ở dấu thập phân), cắt tại khoảng trắng và không cắt giữa ký tự UTF-8, gộp các câu ngắn thành đoạn không quá 150 byte (mục 3.6.4).

```
// ===== HANG DOI NOI: CHIA NHO CAU DAI (Google TTS gioi han ~200 ky tu) =====
#define TTS_CHUNK_MAX    150   // byte, thap hon gioi han cho an toan (tieng Viet co dau 2-3 byte/ky tu)
#define SPEECH_QUEUE_MAX 12

String speechQueue[SPEECH_QUEUE_MAX];
int sqHead = 0, sqTail = 0;
bool speechActive = false;
uint32_t speechStartedAt = 0;

bool sqEmpty() { return sqHead == sqTail; }

void sqPush(const String &s) {
  int next = (sqTail + 1) % SPEECH_QUEUE_MAX;
  if (next == sqHead) {
    Serial.println(" Hang doi noi day, bo doan cuoi");
    return;
  }
  speechQueue[sqTail] = s;
  sqTail = next;
}

// Tach van ban thanh cac doan <= TTS_CHUNK_MAX, uu tien cat o cuoi cau
void enqueueSpeech(const String &text) {
  String buf = "";
  int n = text.length();
  int i = 0;

  while (i < n) {
    // tim cuoi cau (khong cat o dau cham thap phan nhu 3.5)
    int j = i;
    while (j < n) {
      char c = text[j];
      bool isEnd = (c == '!' || c == '?' || c == '\n' || c == ';' ||
                    (c == '.' && !(j > 0 && j + 1 < n && isDigit(text[j - 1]) && isDigit(text[j + 1]))));
      j++;
      if (isEnd) break;
    }
    String sent = text.substring(i, j);
    i = j;
    sent.trim();
    if (sent.length() == 0) continue;

    // cau qua dai: cat tai khoang trang
    while ((int)sent.length() > TTS_CHUNK_MAX) {
```

```
      int cut = sent.lastIndexOf(' ', TTS_CHUNK_MAX);
      if (cut <= 0) {
        cut = TTS_CHUNK_MAX;
        while (cut > 0 && (((uint8_t)sent[cut]) & 0xC0) == 0x80) cut--;   // khong cat giua ky tu UTF-8
        if (cut <= 0) cut = TTS_CHUNK_MAX;
      }
      String part = sent.substring(0, cut);
      part.trim();
      if (buf.length() > 0) { sqPush(buf); buf = ""; }
      if (part.length() > 0) sqPush(part);
      sent = sent.substring(cut);
      sent.trim();
    }
    if (sent.length() == 0) continue;

    // gop cac cau ngan vao mot doan
    if (buf.length() + 1 + sent.length() > TTS_CHUNK_MAX) {
      if (buf.length() > 0) sqPush(buf);
      buf = sent;
    } else {
      buf = (buf.length() > 0) ? (buf + " " + sent) : sent;
    }
  }
  if (buf.length() > 0) sqPush(buf);
}
```

Đoạn mã A.7: Hàm enqueueSpeech chia nhỏ văn bản cho TTS (NCKHa8.ino)

A.8 Gửi lệnh từ giao diện web

Hàm `publishMQTT` và hai hàm điều khiển quạt của web app (mục 3.11).

```
function publishMQTT(topic, payload) {
  if (!topic) return;
  const msgStr =
    typeof payload === "object" ? JSON.stringify(payload) : String(payload);

  if (typeof client !== "undefined" && client && client.connected) {
    client.publish(topic, msgStr);
    addLog(topic, msgStr, "out");
    console.log(" MQTT Gui:", topic, msgStr);
  } else {
    addLog(topic, "[CHUA KET NOI] " + msgStr, "out");
    showToast("Chua ket noi MQTT Broker!", "error");
  }

function toggleFanDevice(devId, isChecked) {
  const dev = state.devices.find((d) => d.id === devId);
  if (!dev) return;
```

```
  dev.isOn = isChecked;
  const payload = isChecked ? `fan_on_${dev.speed || 25}` : "fan_off";
  publishMQTT(dev.pubTopic || "esp32/fan", payload);
  saveState();
  renderDevices();
}

function changeFanSpeed(devId, speed) {
  const dev = state.devices.find((d) => d.id === devId);
  if (!dev) return;
  const selectedSpeed = parseInt(speed, 10);
  if (selectedSpeed === 0) {
    toggleFanDevice(devId, false);
    return;
  }
  dev.speed = selectedSpeed;
  dev.isOn = true;
  publishMQTT(dev.pubTopic || "esp32/fan", `fan_on_${dev.speed}`);
  saveState();
  renderDevices();
}
```

Đoạn mã A.8: Gửi bản tin MQTT và điều khiển quạt trên web (rgb.js)